Artificial Intelligence –
Deep learning 5.2: *Attention & Transformers*

Master in Artificial Intelligence & Computational Engineering

Luis Montesano

Universidad Zaragoza
1542

Today

- Why attention: from RNNs to attention
- Self-attention: values, queries, keys; matrix form
- Positional encoding: sinusoidal, learned, relative
- Text: tokens, embeddings, contextual embeddings
- The transformer block and the encoder stack (BERT)
- Decoders: next-token prediction, causal mask (GPT)
- Encoder–decoder: cross-attention, the original Transformer
- The cost of attention, state-space models; Vision Transformer
- In-class exercise → Lab 5

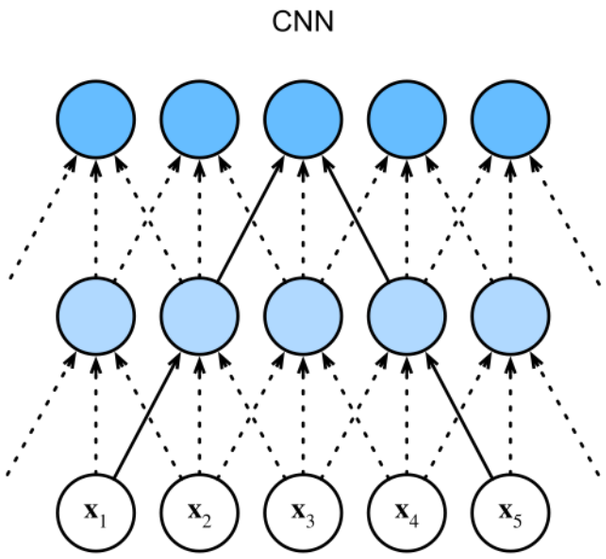
Attention

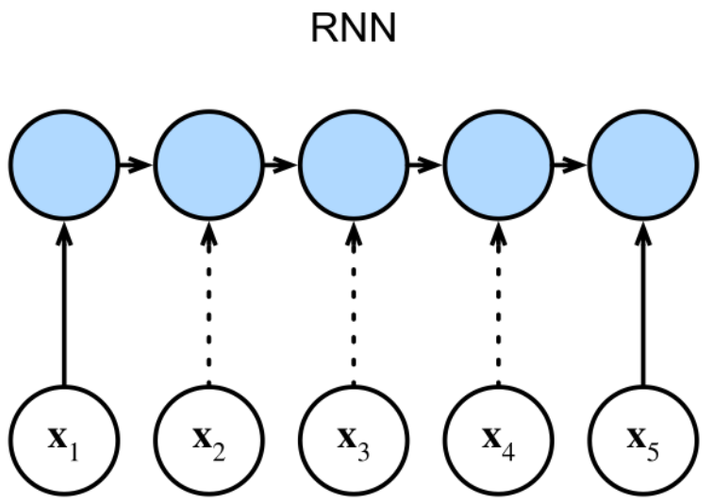
Transformers - Attention

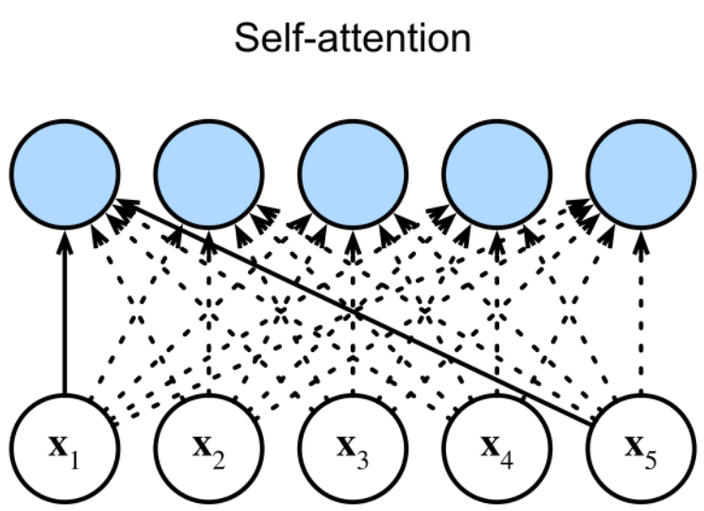


Transformers vs CNN vs RNN

- ***1D CNN - RNN - Self-attention***
  - different tradeoffs speed - expressive power (maximum path length between any two inputs)
  - self-attention: every output is connected to every input
  - Differently from RNN, Transformers can process all input at once (instead of seq.)
  - Differently from CNN, Transformers do not have certain “bias” (as the assumptions encoded in CNNs about *nearby* more important than *further away*)



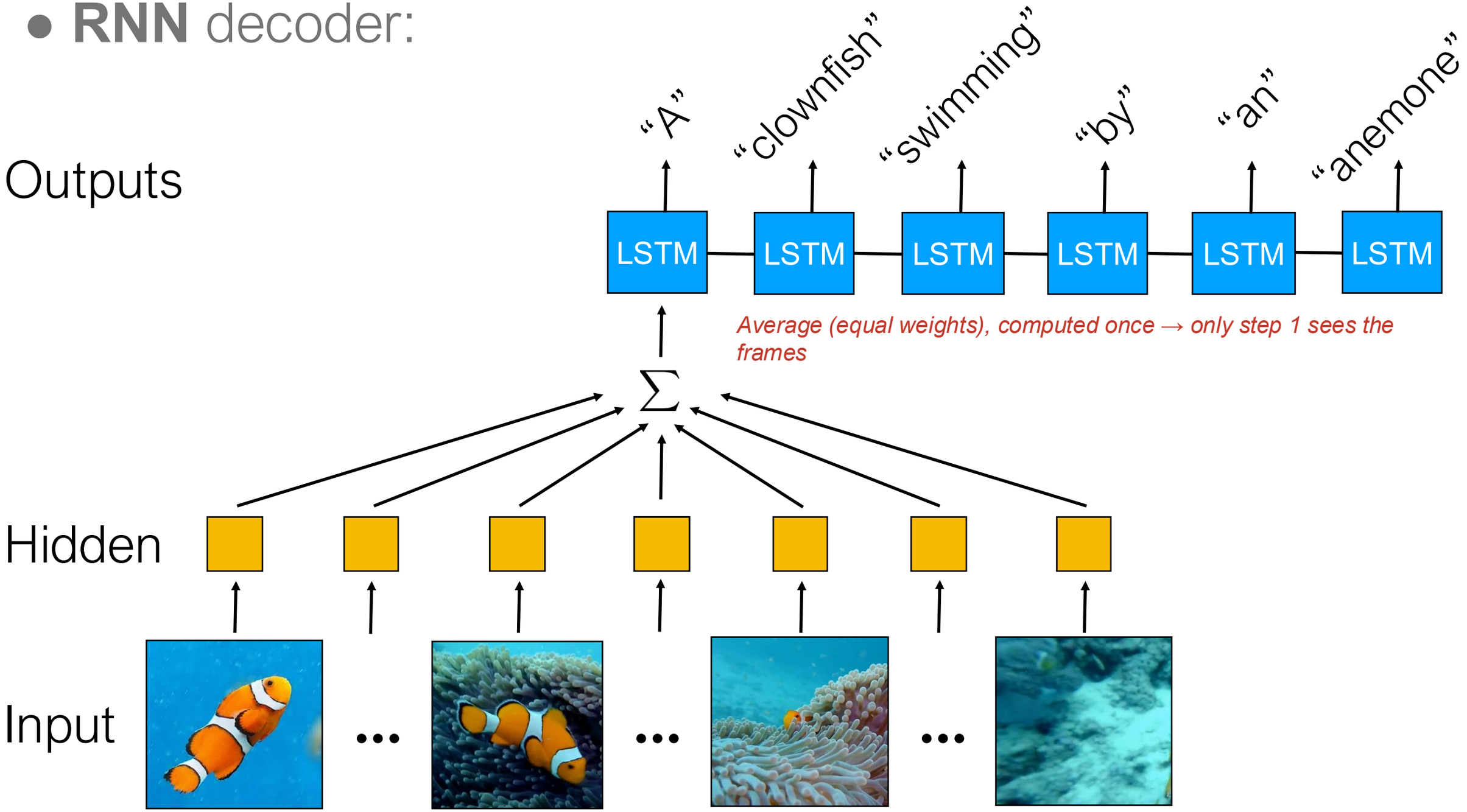




A. Zhang, Z. Lipton, M. Li, and A. Smola. Dive into deep learning. 2020.

RNN vs Attention

- **RNN** decoder:

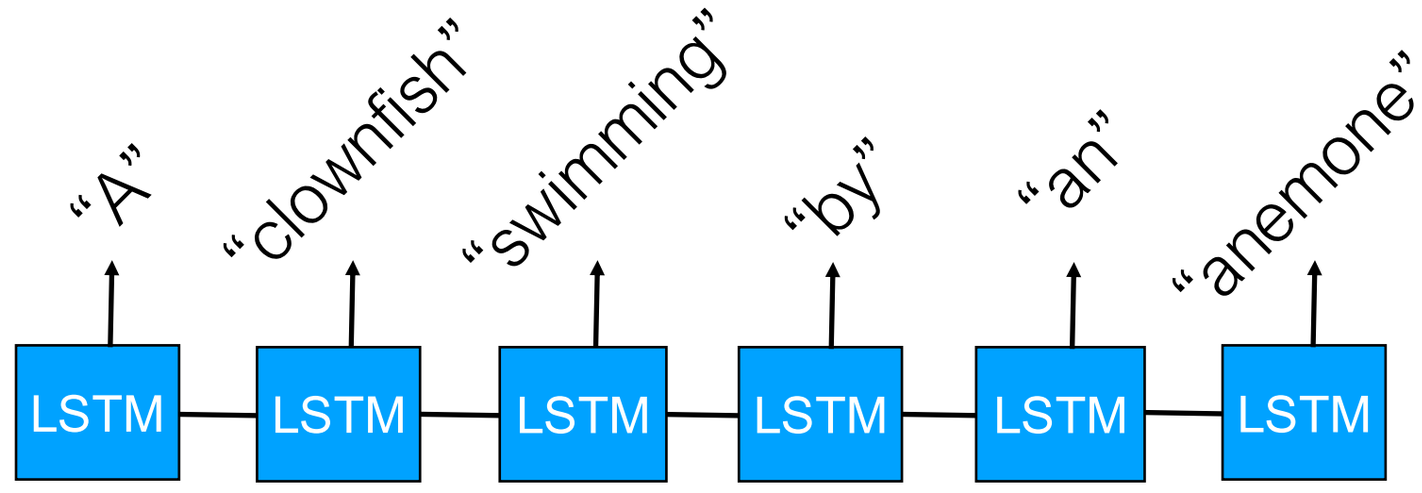


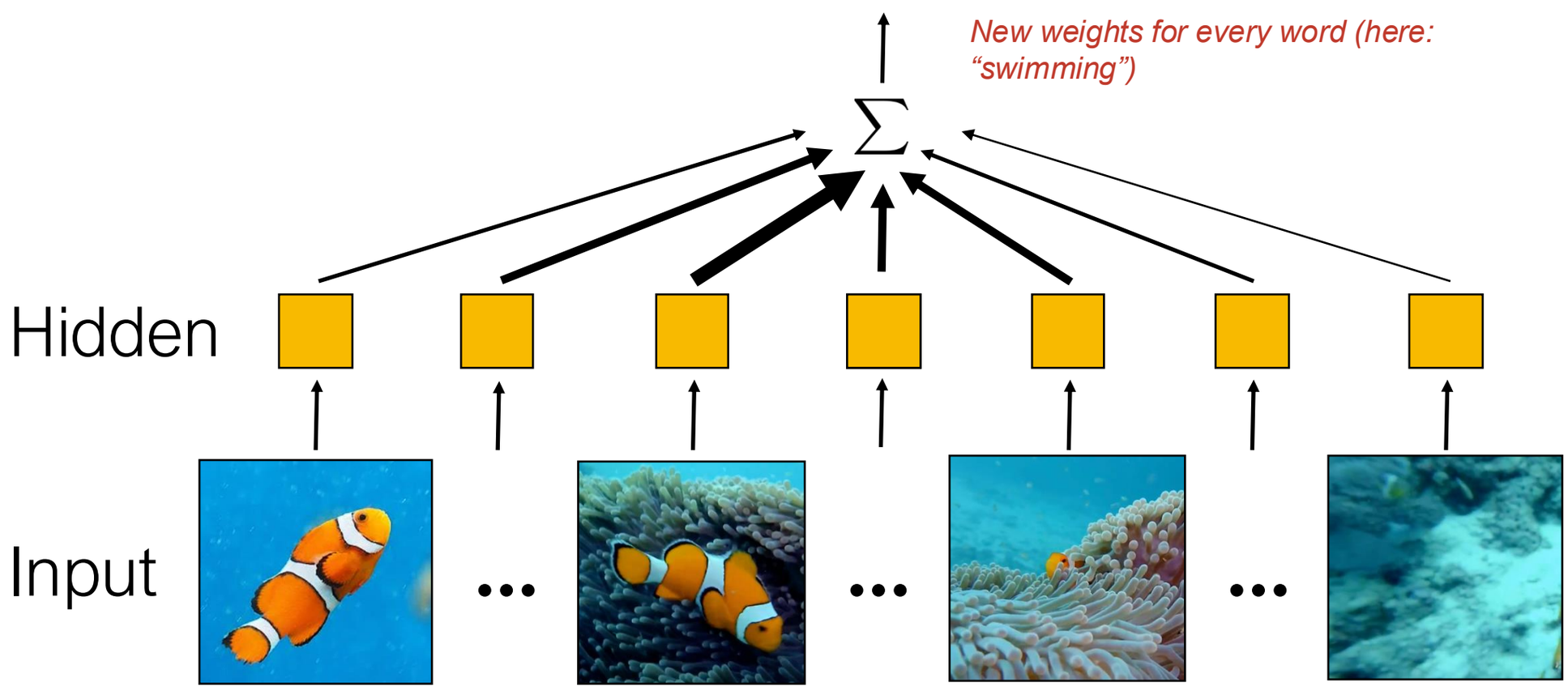
slides from: B. Freeman. P. Isola. *Advances in Computer Vision. MIT. 2021*

RNN vs Attention

- **Attention**:

Outputs

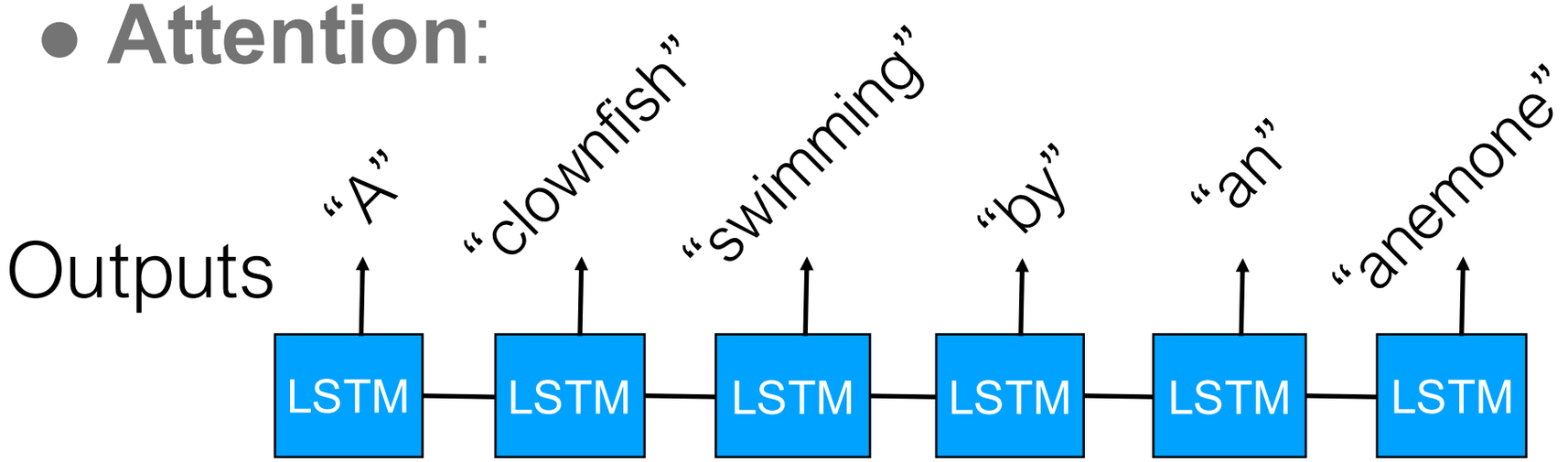


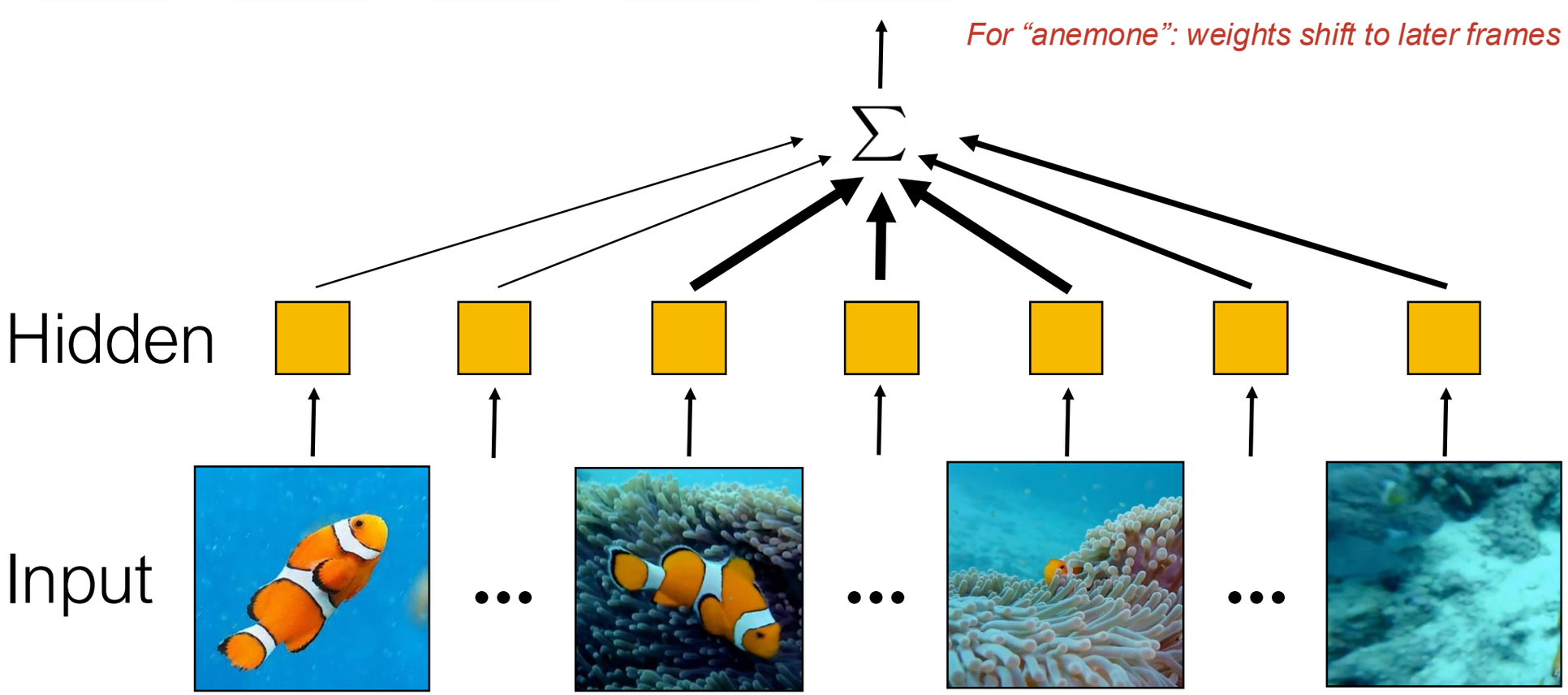


slides from: B. Freeman. P. Isola. *Advances in Computer Vision. MIT. 2021*

RNN vs Attention

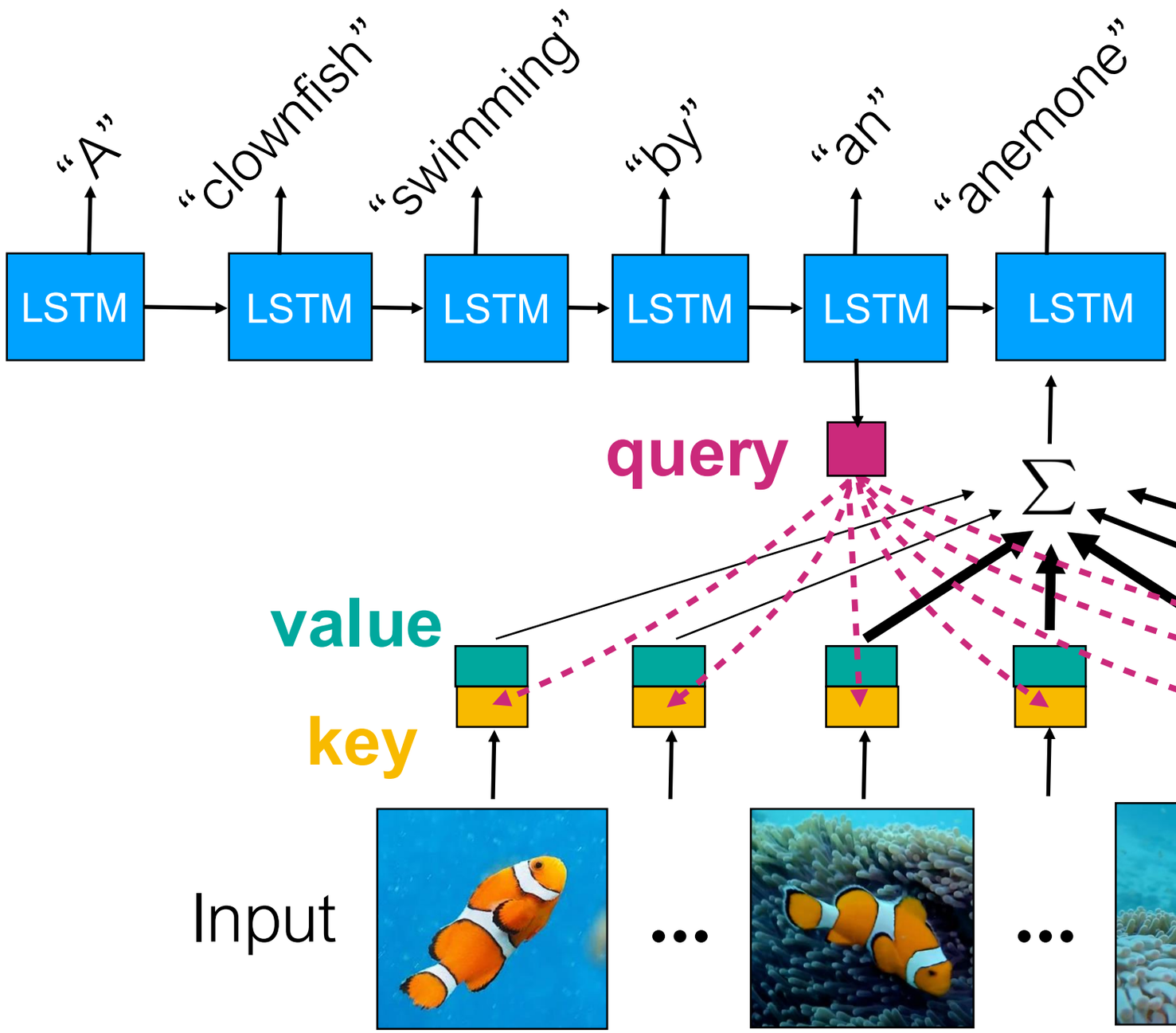
- **Attention**:



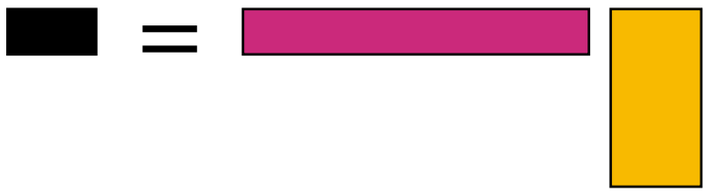


slides from: B. Freeman. P. Isola. *Advances in Computer Vision. MIT. 2021*

Transformers vs RNN



weights are determined by similarity between **query** and **key**



summation is over **weights** * value

slides from: B. Freeman. P. Isola. *Advances in Computer Vision. MIT. 2021*

Self-attention (1): weighted sums

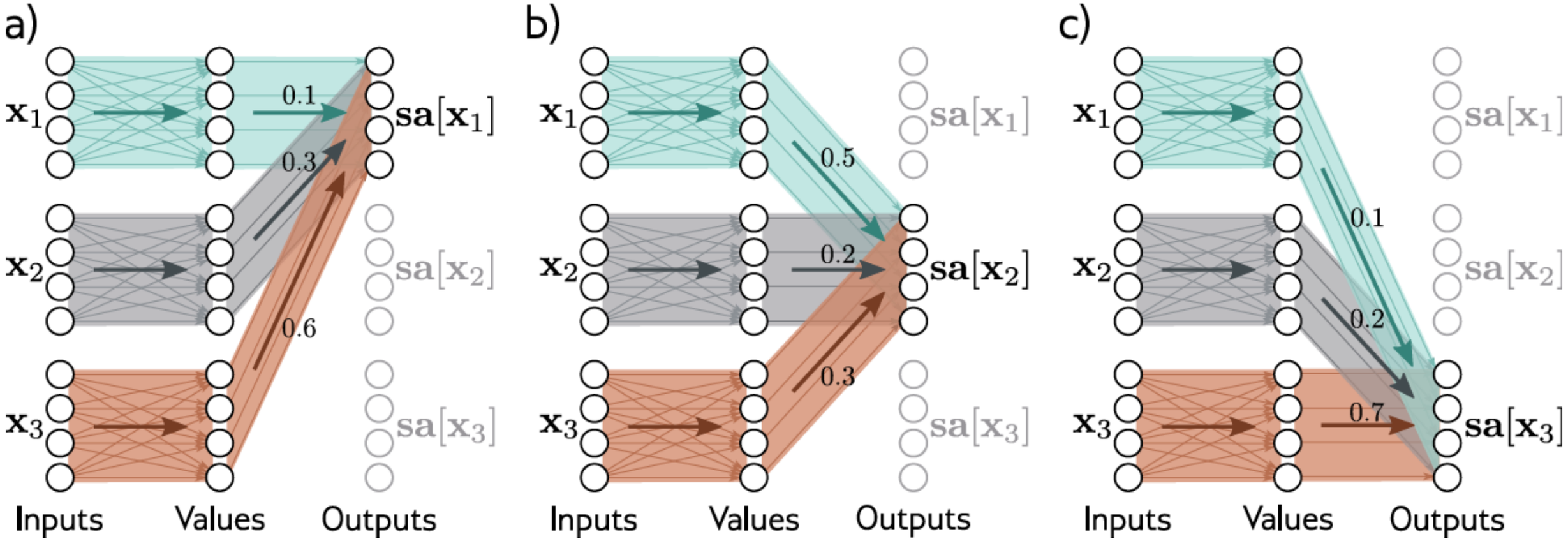
- **N inputs $\mathbf{x}_1$ … $\mathbf{x}_N$**, each a D-dim vector (a frame feature, a word embedding, …)
- Each input is mapped to a **value**: the same linear map for every input, no ReLU

$$\mathbf{v}_m = \mathbf{W}_V \mathbf{x}_m$$

- Each **output** is a weighted sum of **all** the values:

$$\mathbf{sa}_n = \sum_{m=1}^{N} a_{nm}\,\mathbf{v}_m \qquad a_{nm} \geq 0, \quad \sum_m a_{nm} = 1$$

- $\mathbf{a}_{nm}$: share of input m in output n, **computed from the inputs** (next slide)



Figures and notation adapted from S. J. D. Prince. Understanding Deep Learning. MIT Press, 2023, ch. 12 (udlbook.github.io)

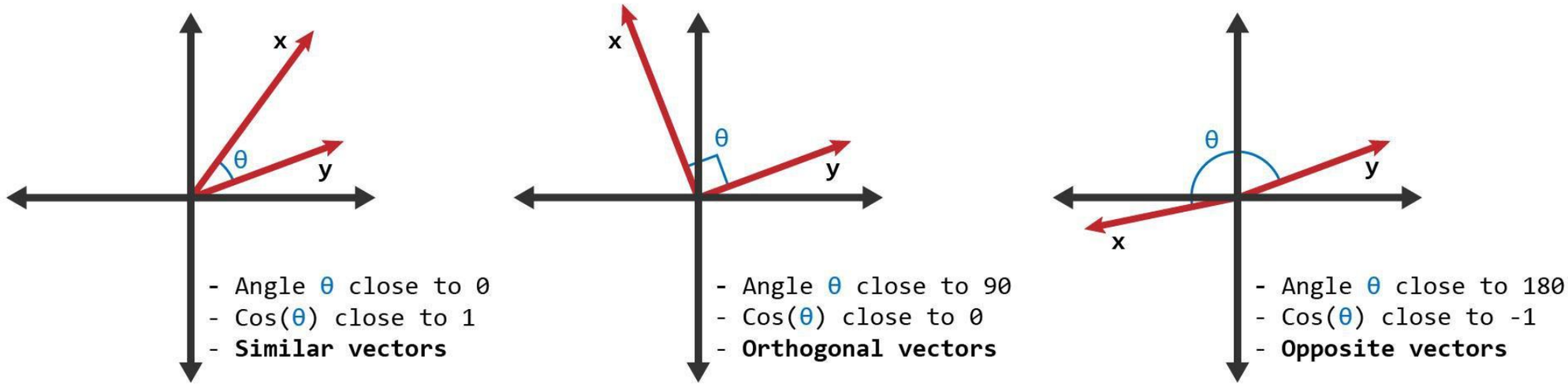
Self-attention (2): the weights

- Each input also produces a **query** and a **key** (two more learned linear maps):

$$\mathbf{q}_n = \mathbf{W}_Q \mathbf{x}_n \qquad \mathbf{k}_m = \mathbf{W}_K \mathbf{x}_m$$

- Score = **key · query** (similarity); softmax over m → weights of output n:

$$a_{nm} = \mathrm{softmax}_m(\mathbf{k}_m^\mathsf{T}\mathbf{q}_n) = \frac{\exp(\mathbf{k}_m^\mathsf{T}\mathbf{q}_n)}{\sum_{m'=1}^{N}\exp(\mathbf{k}_{m'}^\mathsf{T}\mathbf{q}_n)}$$



Same roles as slide 8 — query: what output n needs · key: what input m offers. New: **every input is also a query** → self-attention.

Figures and notation adapted from S. J. D. Prince. Understanding Deep Learning. MIT Press, 2023, ch. 12 (udlbook.github.io)

Self-attention (3): a worked example

- N = 3 inputs, D = 2, already projected. Compute **output 1**, whose query is $\mathbf{q}_1 = (1, 0)$

m	key k_m	score $k_m^T q_1$	weight a_{1m}	value v_m
1	(1, 0)	1	0.245	(1, 0)
2	(0, 1)	0	0.090	(0, 2)
3	(2, 1)	2	**0.665**	(3, 1)

softmax(1, 0, 2) = (0.245, 0.090, 0.665)

$$\mathbf{sa}_1 = 0.245\,\mathbf{v}_1 + 0.090\,\mathbf{v}_2 + 0.665\,\mathbf{v}_3 = (2.24,\ 0.85)$$

- Key 3 matches q_1 best → input 3 dominates output 1

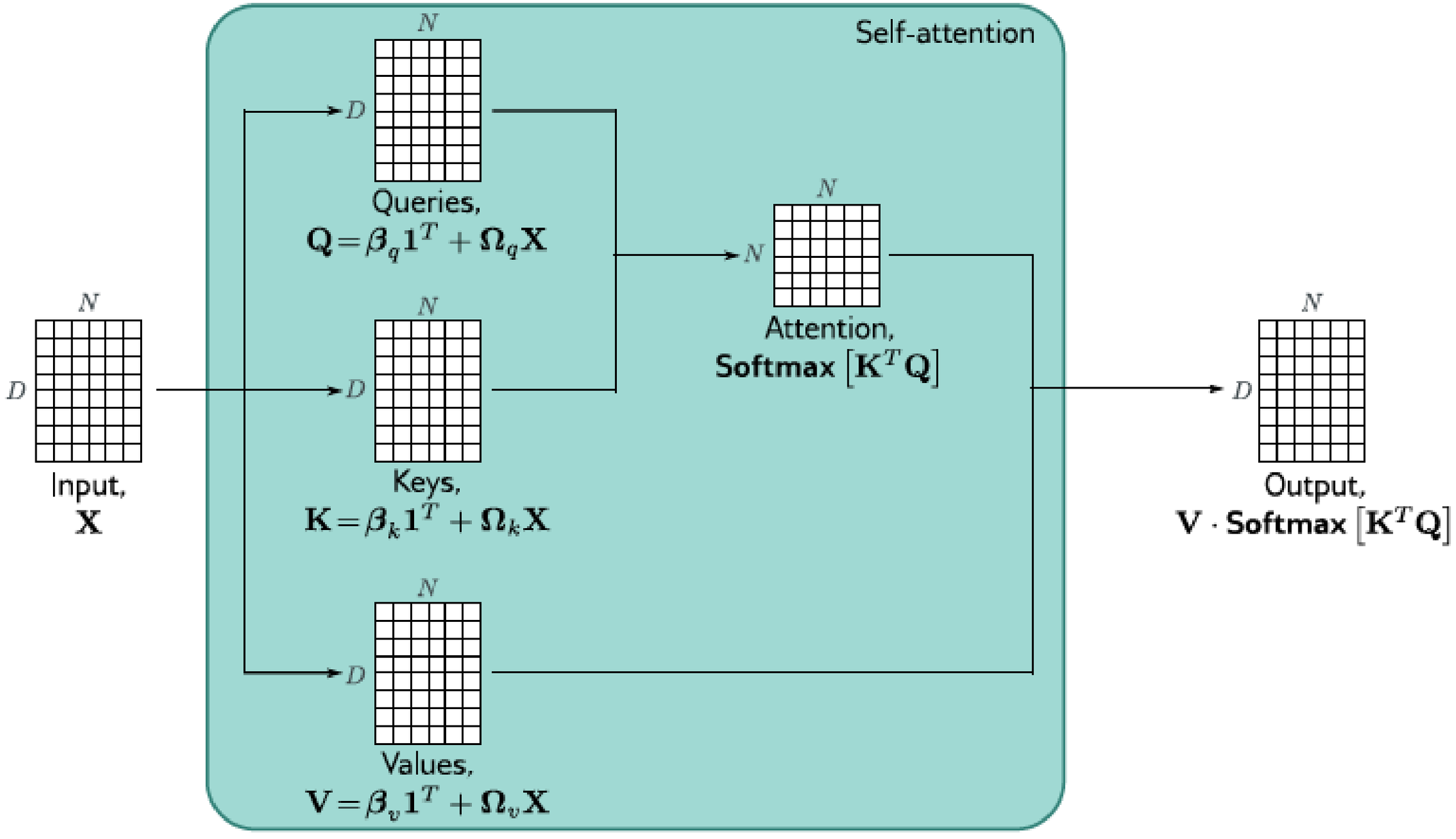
- Outputs 2 and 3: same computation with q_2, q_3 → independent → **all in parallel** (matrix form, later)

Key properties

- Parameters: only the three projections (values, queries, keys). They do not grow with N → any sequence length
- Weights are computed from the input, not stored: a data-dependent layer
- Every output reaches every input in one step
- Order-blind: shuffle the inputs → outputs shuffled the same way (→ positional encodings)
- Cost: N × N scores

Toy numbers for illustration. Notation: S. J. D. Prince. Understanding Deep Learning. MIT Press, 2023, ch. 12

Self-attention in matrix form



Figures and notation adapted from S. J. D. Prince. Understanding Deep Learning. MIT Press, 2023, ch. 12 (udlbook.github.io)

Self-attention in matrix form

- Inputs: the N token embeddings as rows of X ∈ $\mathbb{R}^{N\times D}$. Three learned projections:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- QK^T: N × N scores, every query against every key; softmax **row by row** → weights that sum to 1
- Output: N × d_v, each row a weighted average of the values
- Why $\sqrt{d_k}$? If the components of q and k are independent with zero mean and unit variance, q·k has variance d_k. Dividing keeps the scores O(1), so the softmax does not saturate (tiny gradients)

A. Vaswani et al. Attention is all you need. NeurIPS 2017

Self-attention: just matrix multiplications

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

- Attention *is just* really large matrix multiplications!
- Many operations …
- … but super parallelizable on GPUs/TPUs

Positional encoding (1): why we need it

- Self-attention is **order-blind**: permute the inputs → outputs permuted the same way
- "The man ate the fish" vs "The fish ate the man": same tokens → **same outputs**
- **Idea:** give each position n a vector $\mathbf{p}_n$ and **add** it to the token embedding:

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{p}_n \qquad \tilde{\mathbf{X}} = \mathbf{X} + \mathbf{P}$$

token	"The"	"fish"	"ate"	"the"	"man"
token embedding	x(the)	x(fish)	x(ate)	x(the)	x(man)
	+	+	+	+	+
position encoding	$\mathbf{p}_1$	$\mathbf{p}_2$	$\mathbf{p}_3$	$\mathbf{p}_4$	$\mathbf{p}_5$
	=	=	=	=	=
input to attention	$\tilde{\mathbf{x}}_1$	$\tilde{\mathbf{x}}_2$	$\tilde{\mathbf{x}}_3$	$\tilde{\mathbf{x}}_4$	$\tilde{\mathbf{x}}_5$

In "The man ate the fish", 'man' gets p_2 instead of p_5 → a different input, different queries, keys and values.
Attention now sees what a token is and where it is.

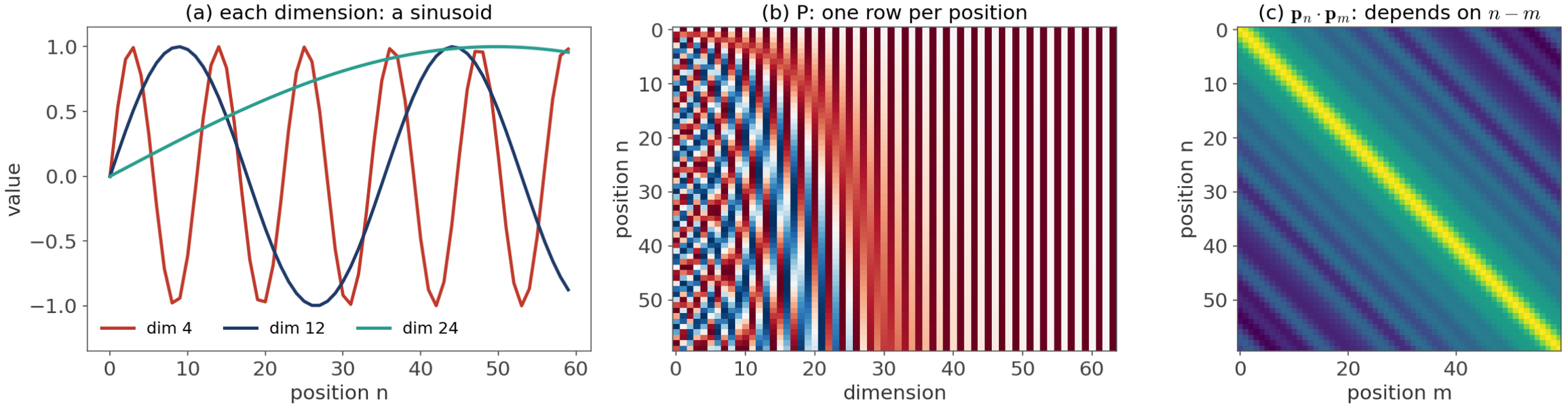
Example after S. J. D. Prince. Understanding Deep Learning. MIT Press, 2023, ch. 12

Positional encoding (2): fixed sinusoids

- Why not p_n = n? Unbounded. Or n / N? Depends on the length.
- **Vaswani et al.: sines and cosines at different frequencies** (no parameters):

$$P_{n,2i} = \sin\left(\frac{n}{10000^{2i/D}}\right) \qquad P_{n,2i+1} = \cos\left(\frac{n}{10000^{2i/D}}\right)$$

- Each (sin, cos) pair is a **clock**: fast in the first dims, slow in the last



- Works for any length; p_{n+k} is a rotation of p_n → offsets are easy to read with dot products (c)

Positional encoding (3): learned, relative

- **Learned absolute:** a trainable table P (N_{max} × D), like the token embeddings
  - simple, but undefined beyond N_{max}. BERT, GPT-2, ViT
- **Relative:** what matters is the offset n − m → put it in the score
  - RoPE (rotary): rotate q_n by angle ∝ n, k_m by angle ∝ m → $q_n \cdot k_m$ depends on content and n − m. Every layer. Llama, Mistral, Qwen
  - ALiBi: subtract a penalty ∝ |n − m| from the score (BLOOM)

	Parameters	Where	Beyond training length	Used in
Sinusoidal (fixed)	none	added to the input	defined, limited in practice	original Transformer
Learned absolute	N_{max} × D	added to the input	no	BERT, GPT-2, ViT
Rotary (RoPE)	none	rotates q, k in every layer	better (with scaling tricks)	most current LLMs

For images: 2D positions (row, column) of each patch → Vision Transformer, later

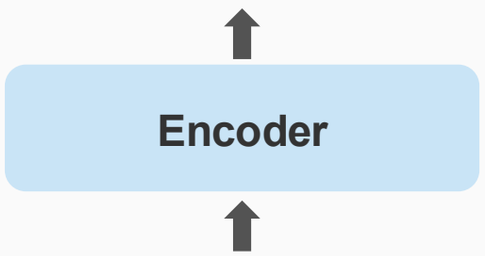
J. Devlin et al. BERT. NAACL 2019 · J. Su et al. RoFormer (RoPE). 2021 · O. Press et al. ALiBi. ICLR 2022

Where we are going: transformers for text

Same building block, **three ways to use it**: what goes in, what comes out

Encoder-only
e.g. BERT

label: “positive”
(or one vector per token)



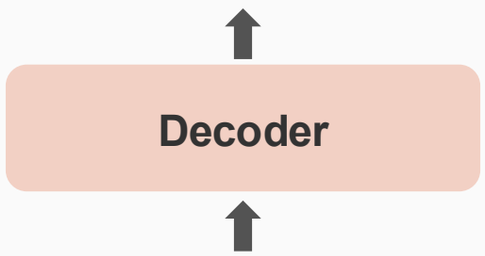
“The movie was great”

Tasks

sentiment, topic, spam
named entities
search / embeddings
(images: ViT)

Decoder-only
e.g. GPT, Llama, Claude

next token: “the”
append it, repeat → text



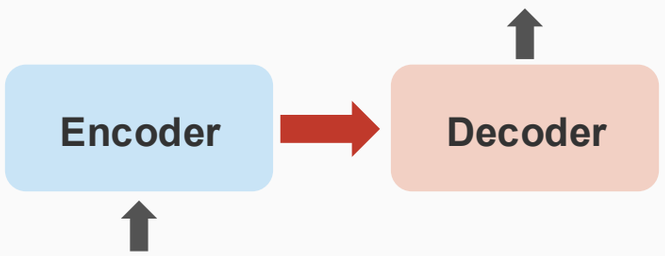
“The fish lived in”

Tasks

text generation
chat assistants
code completion

Encoder–decoder
“Attention is all you need”, T5

“I am a student”
(one token at a time)



“Je suis étudiant”

Tasks

translation
summarisation
captioning, speech → text

Our route through the next slides:

Text → vectors → **Encoder** → **Decoder** → **Encoder–decoder**

Vaswani et al. 2017 · Devlin et al. (BERT) 2019 · Radford et al. (GPT) 2018 · Raffel et al. (T5) 2020

Tokenisation: from text to tokens

- A transformer processes a sequence of **tokens** from a fixed vocabulary
- Words as tokens?
  - unknown words (names, typos), punctuation
  - walk, walks, walked, walking: unrelated tokens
- Characters? tiny vocabulary, very long sequences
- **Sub-word** tokenisation, e.g. Byte-Pair Encoding: start from characters, repeatedly **merge the most frequent pair**
  - frequent words = 1 token; rare words = several pieces
  - typical vocabularies: 30k–200k tokens

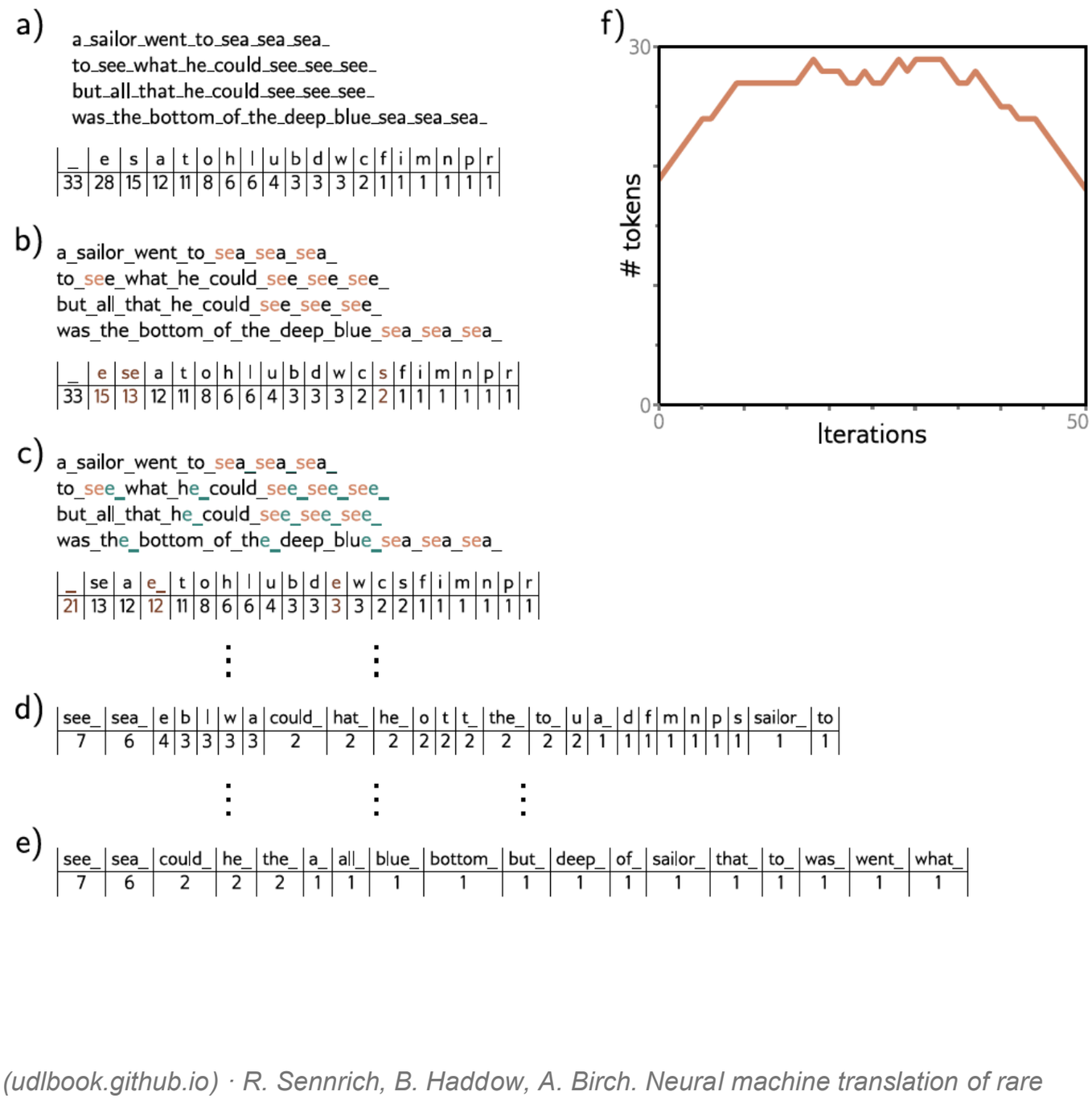
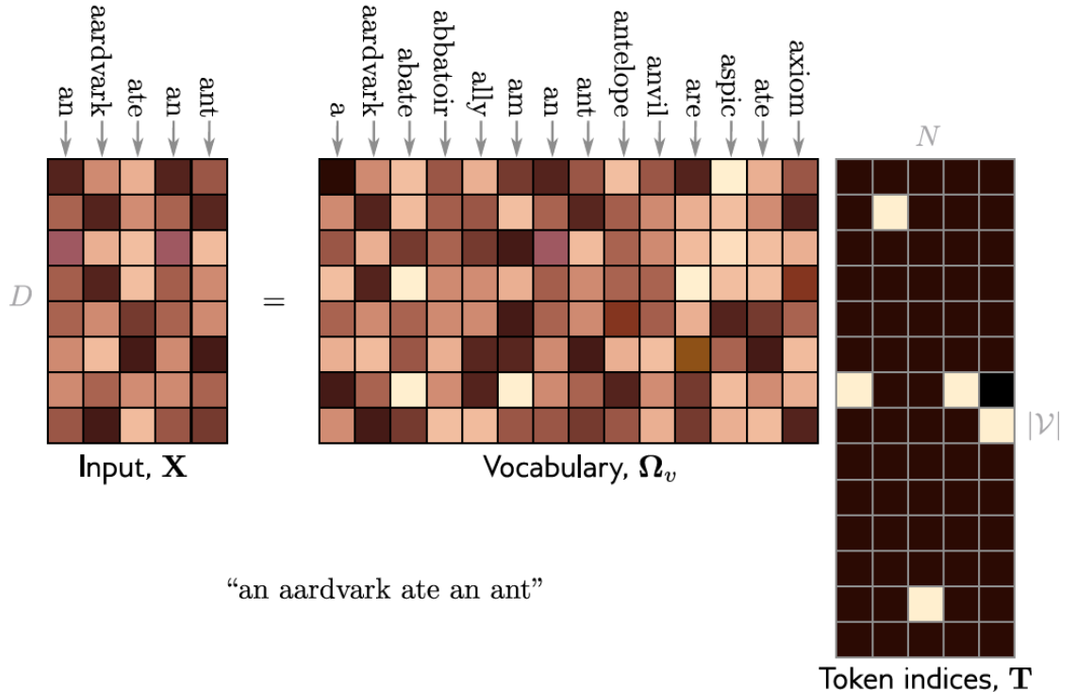


Figure: S. J. D. Prince. Understanding Deep Learning. MIT Press, 2023, ch. 12 (udlbook.github.io) · R. Sennrich, B. Haddow, A. Birch. Neural machine translation of rare words with subword units. ACL 2016

From tokens to embeddings

- Each token index → one-hot vector t_n of size |V| (vocabulary)
- A **learned** embedding matrix Ω maps it to a D-dimensional vector:

$$x_n = \Omega\, t_n, \qquad \Omega \in \mathbb{R}^{D \times |\mathcal{V}|} \quad \text{(a lookup table: column } t_n \text{ of } \Omega\text{)}$$



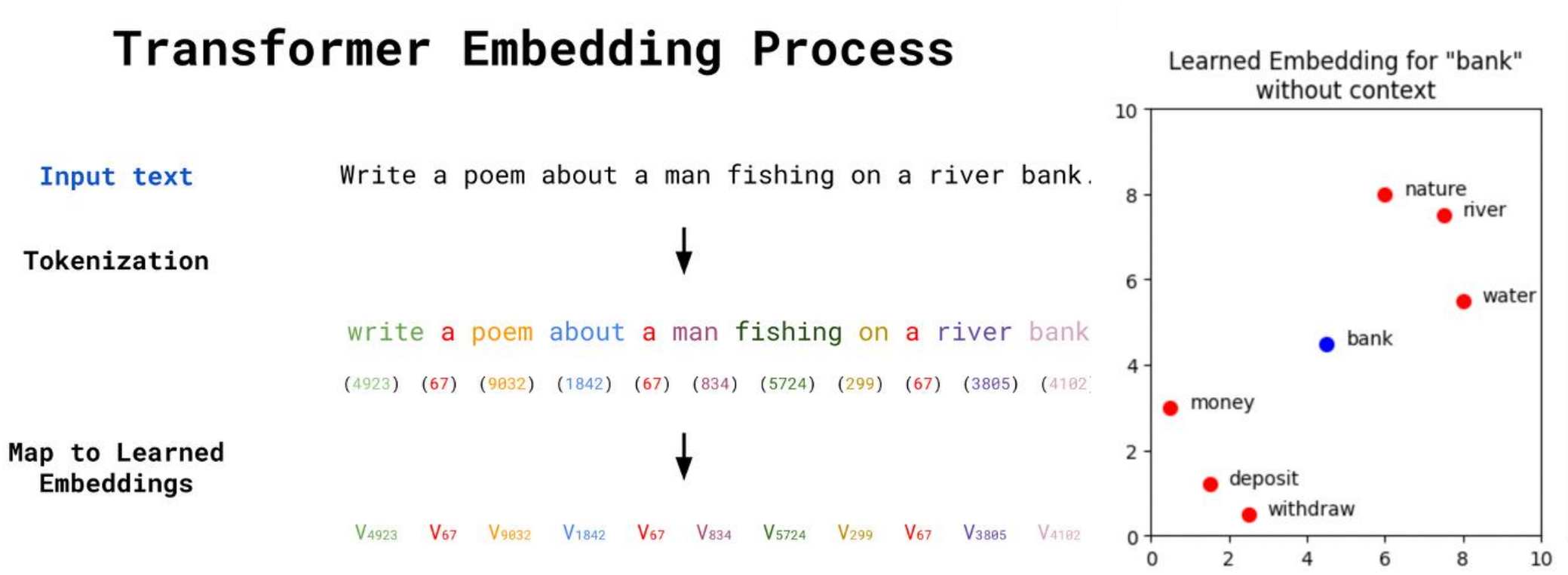
Ω is trained with the rest of the network (nn.Embedding)

Stack the N embeddings as rows: X $\in \mathbb{R}^{N \times D}$

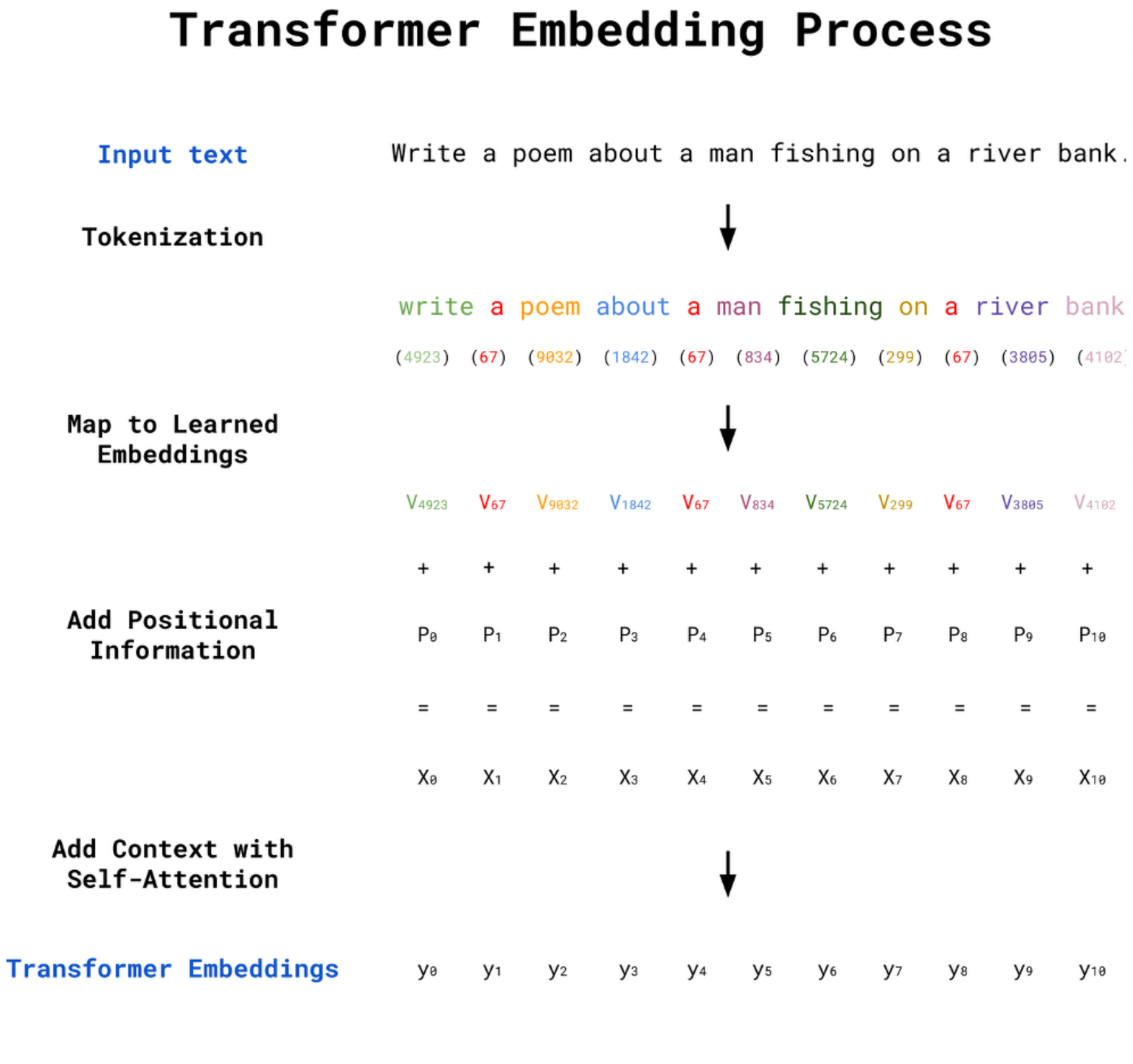
Context-free: 'bank' (river) = 'bank' (money). Self-attention will fix that

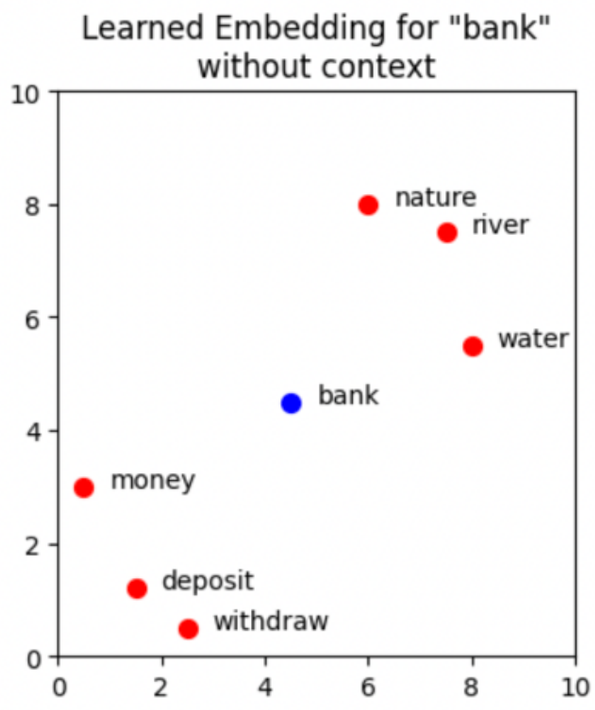
Figure: S. J. D. Prince. Understanding Deep Learning. MIT Press, 2023, ch. 12 (udlbook.github.io)

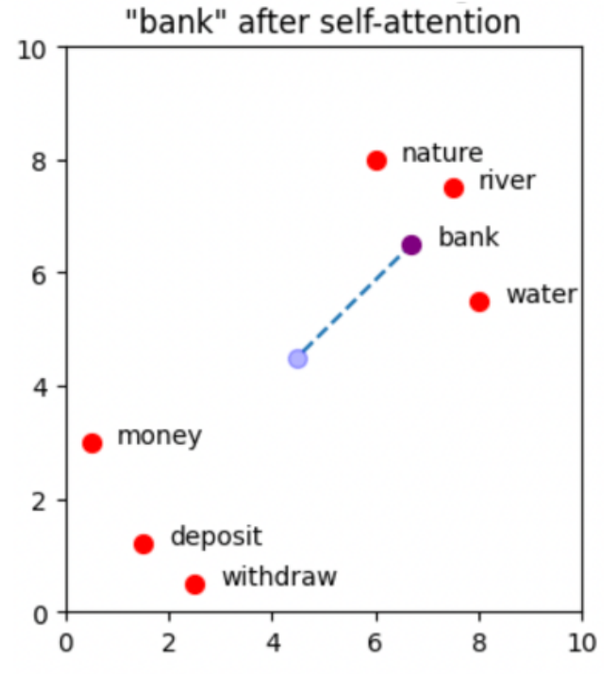
Text → tokens → embeddings



Contextual embeddings







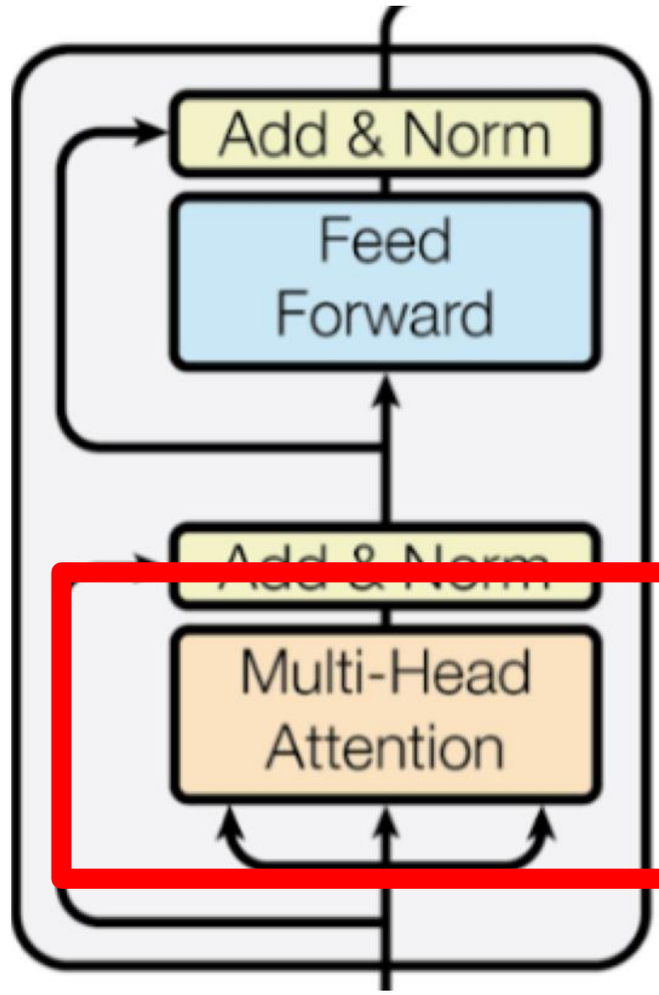
The transformer block

GOAL: augment each token representation based on relationship to other tokens

One block, four steps:

- Multi-head self-attention: tokens exchange information
- Add & Norm: residual connection + layer normalisation
- Feed-forward: the same MLP on each token separately
- Add & Norm

Stack N blocks (input and output: N × D)



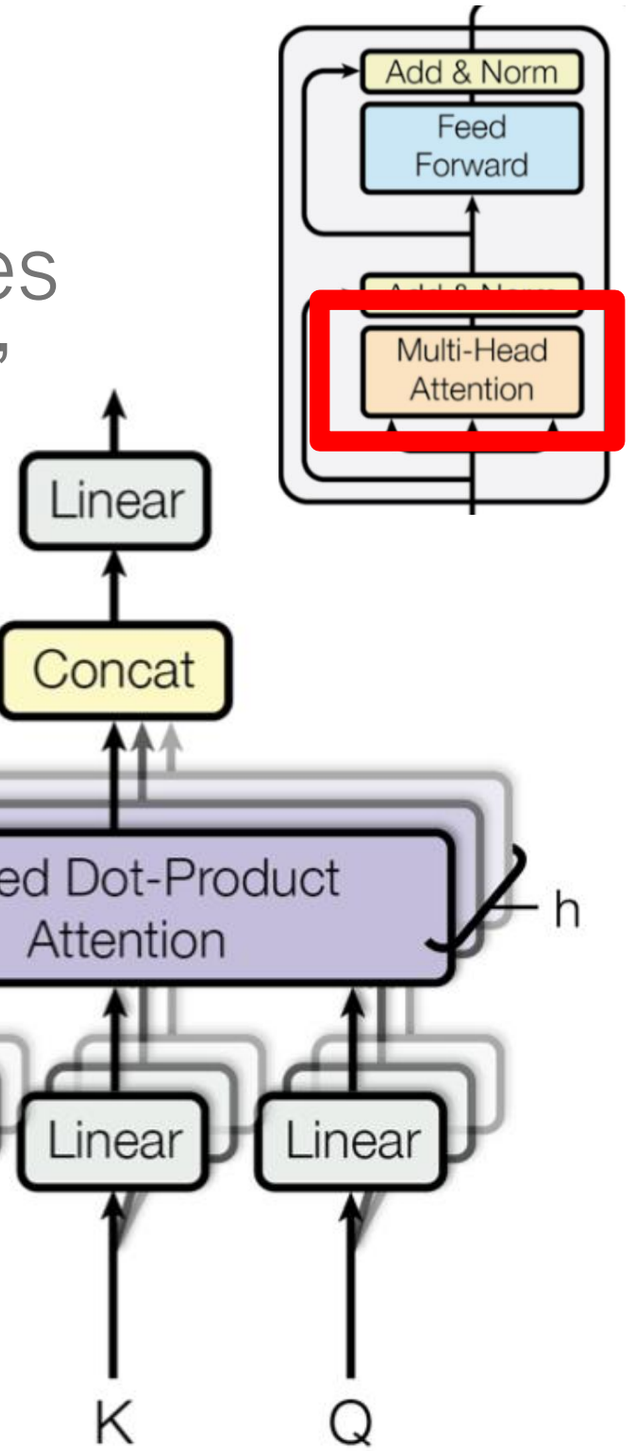
Transformer/attention block

A. Vaswani et al. “Attention Is All You Need”. In: NIPS. 2017.

Block (1): multi-head attention

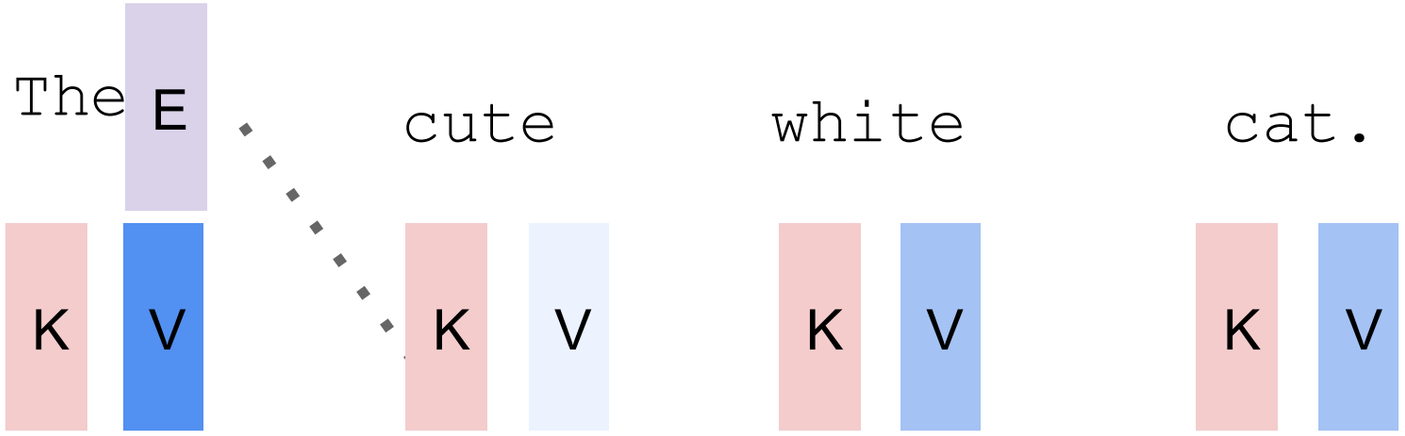
Multi-head Attention (h “heads”):

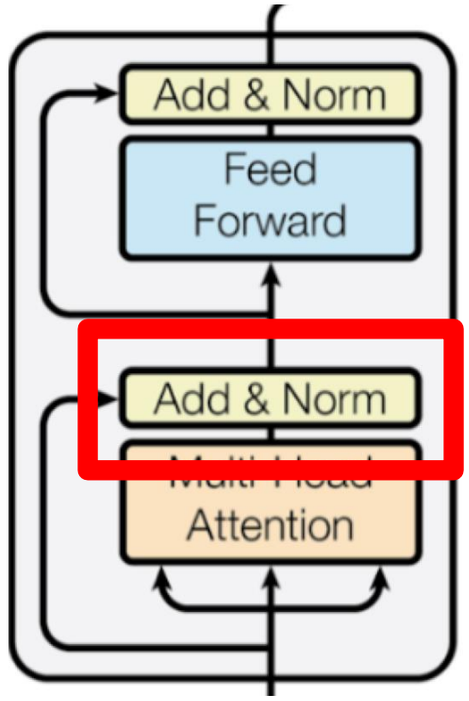
- Use different sets of projection matrices (W_K, W_Q and W_V), one per “head”
- Process each “head”, in parallel
- Concatenate all h values and re-project them to get the final embedding

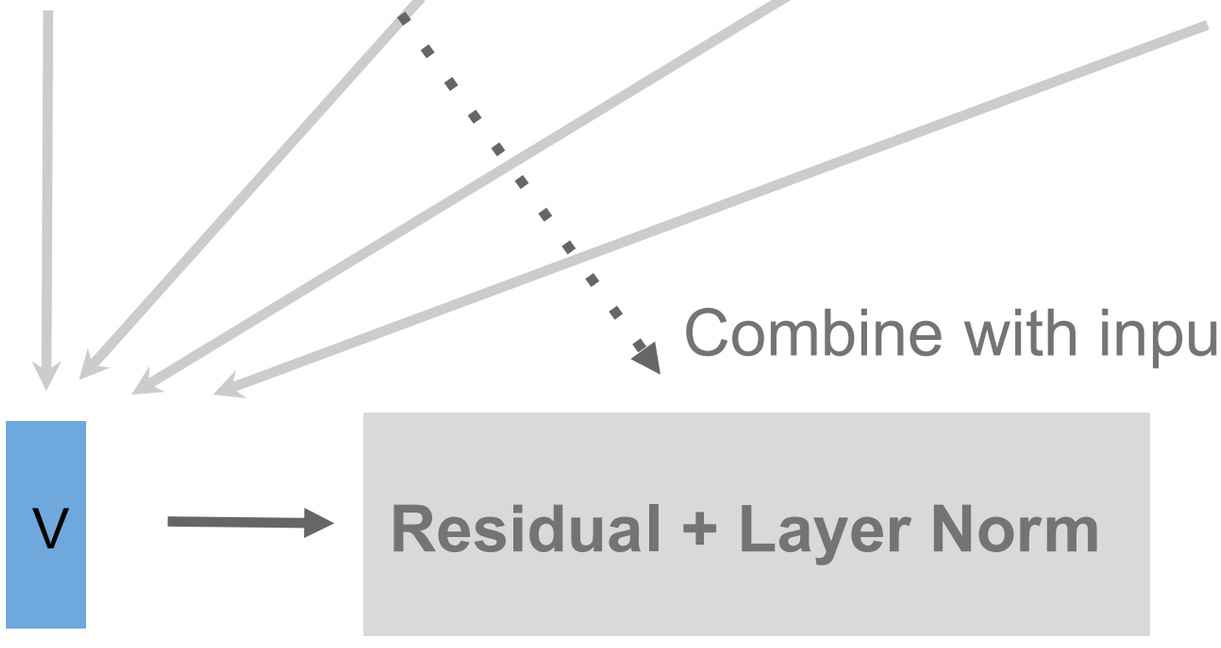


Block (2): Add & Norm

Add & Norm: Residual connection
+ Layer Normalization*





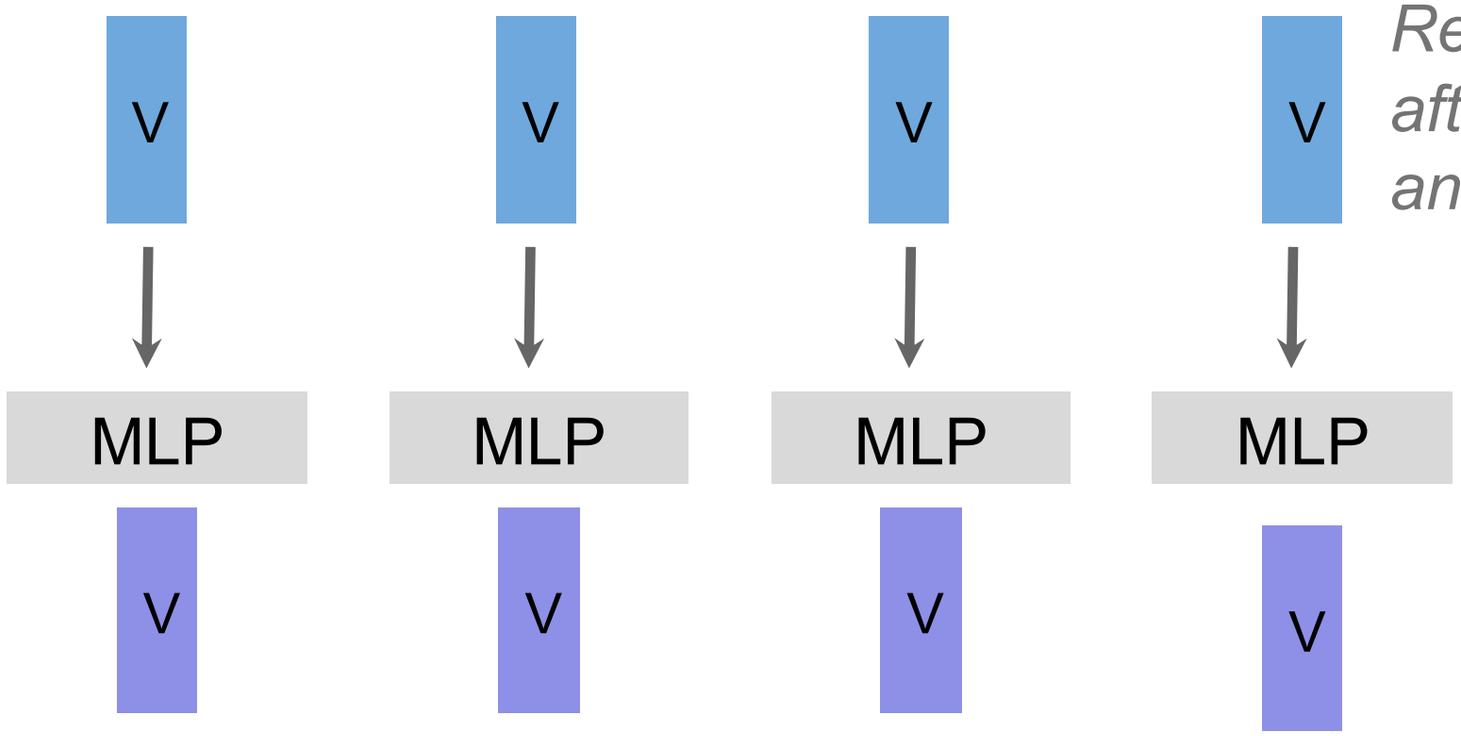


Combine with input embeddings (Residual)

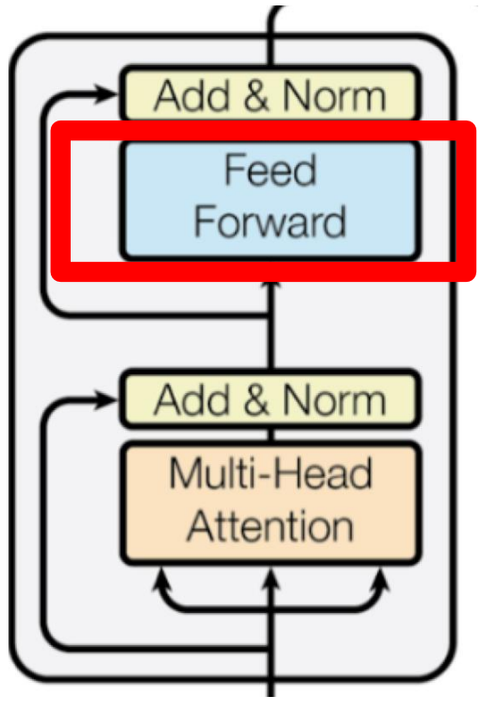
* batch normalization with statistics computed for each sample

Block (3): feed-forward

Feed Forward: Two layer MLPs applied individually to each token embedding



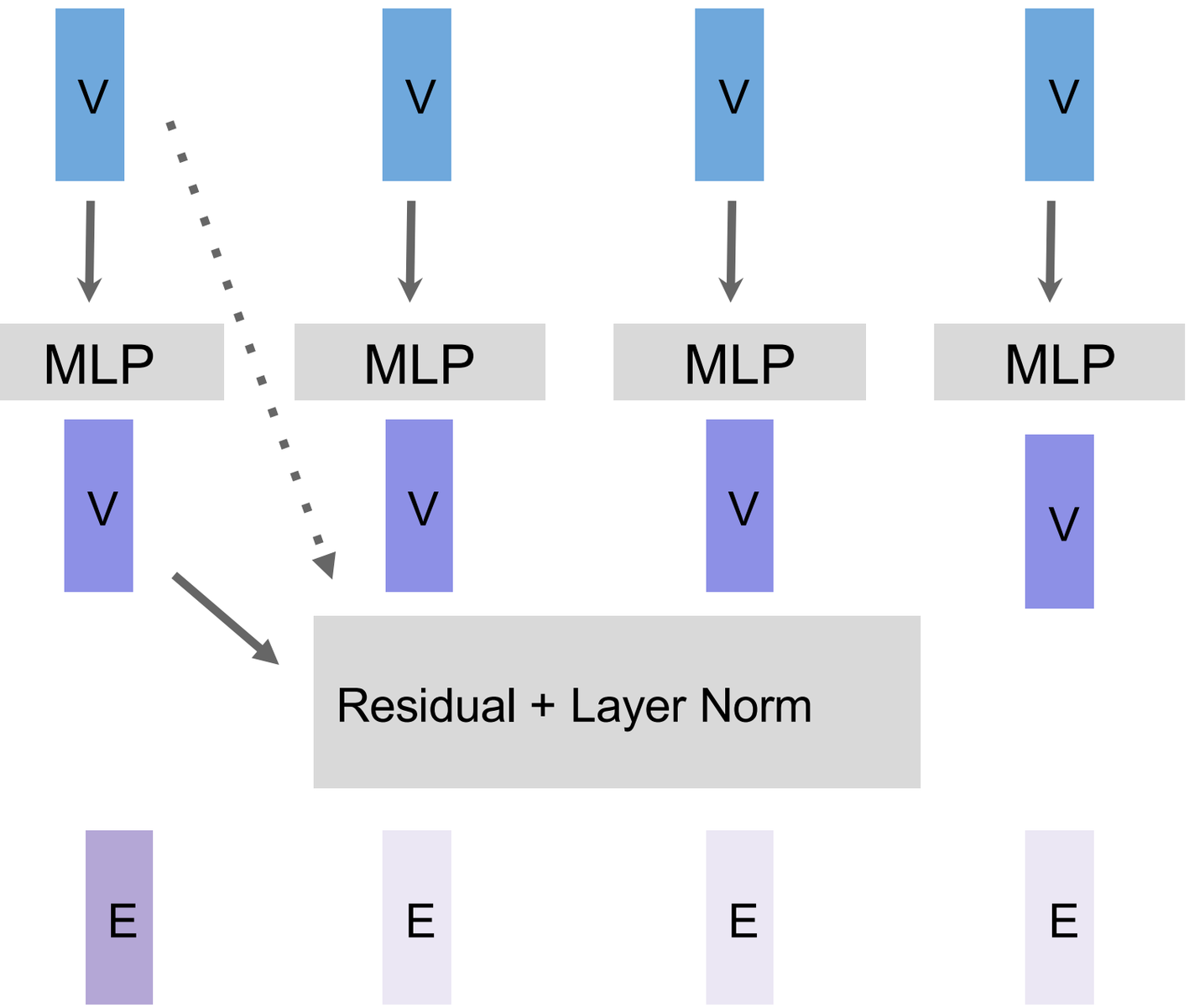
Representations after Multi-Head and Add&Norm

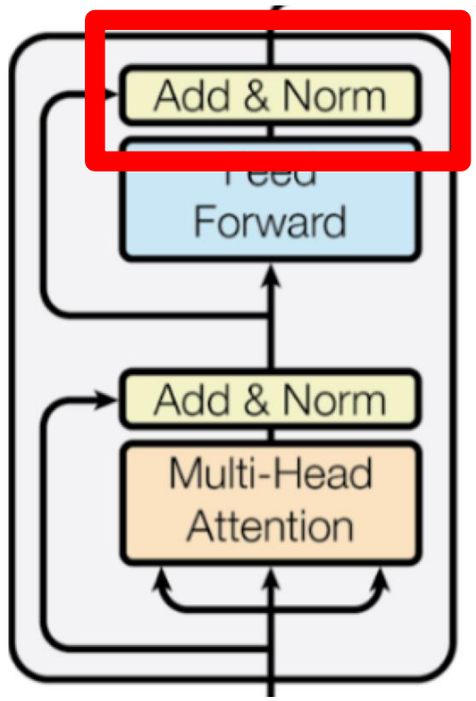


- Attention: “recombining” input vectors
- MLP: adds non-linearities and linear layers to “mix” the features of each token

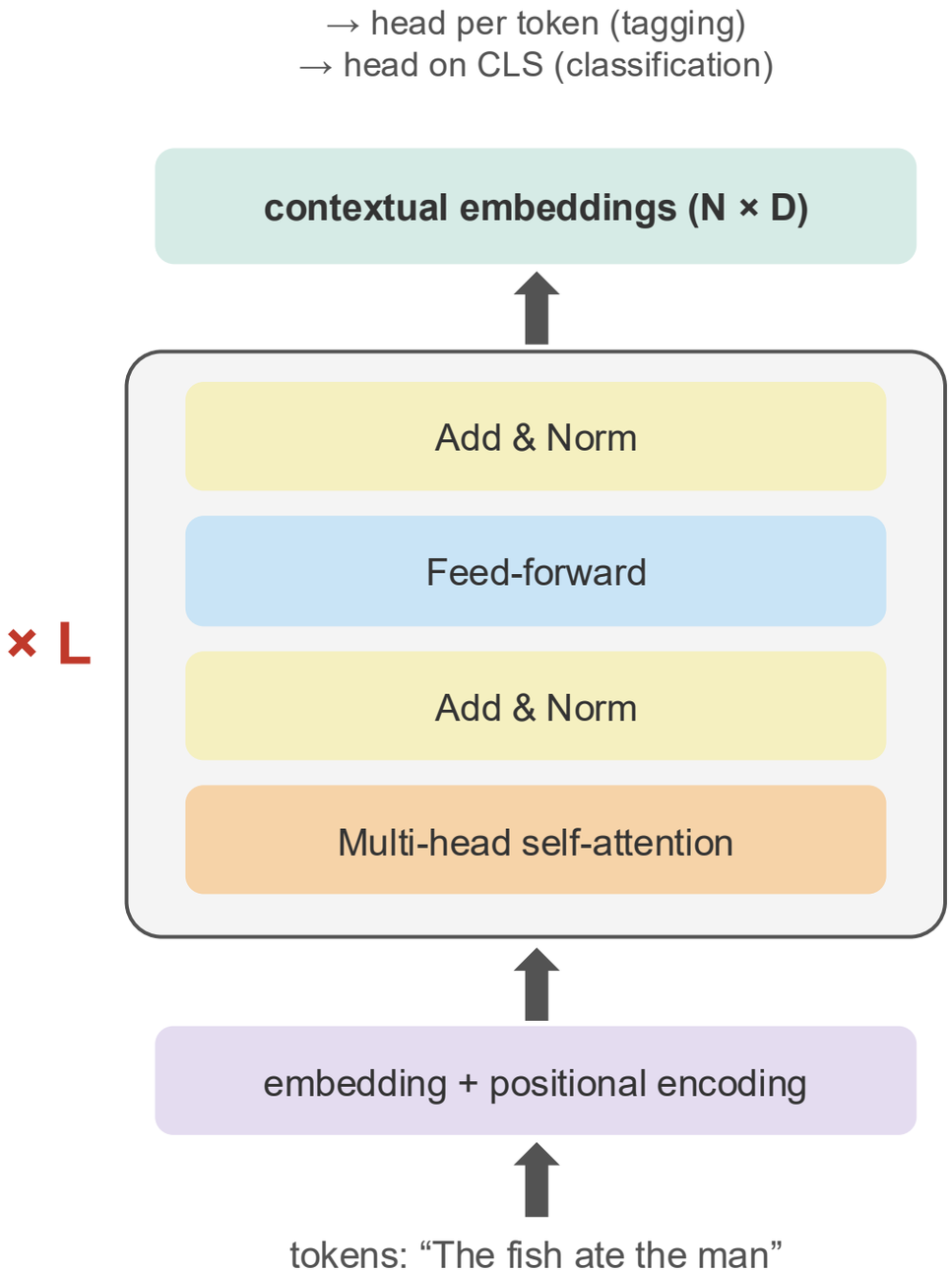
Block (4): Add & Norm

Add & Norm:





Stacking blocks: the transformer encoder



- Each block maps N × D → N × D, so blocks can be **stacked**: L blocks, each with its own weights
- L = 6 in the original paper, 12 in BERT-base, ~100 in large LLMs
- **No mask**: every token sees every other token, in every layer (bidirectional)
- Output: one contextual vector per token, used by a task head (next slide)
- Pre-training (BERT): hide ~15 % of the tokens and predict them from both sides (**masked language modelling**)
- The same encoder works on image patches: ViT (end of the lecture)

In the original paper's figure this stack is the “N×”. We write L: N is the number of tokens.

A. Vaswani et al. NeurIPS 2017 · J. Devlin et al. BERT. NAACL 2019

Using the encoder: the CLS token

CLS Token

- Often have a dummy token attached to beginning of input
- Use the output representation of that token for classification/other downstream tasks (e.g. pass it into a feed forward network to predict)

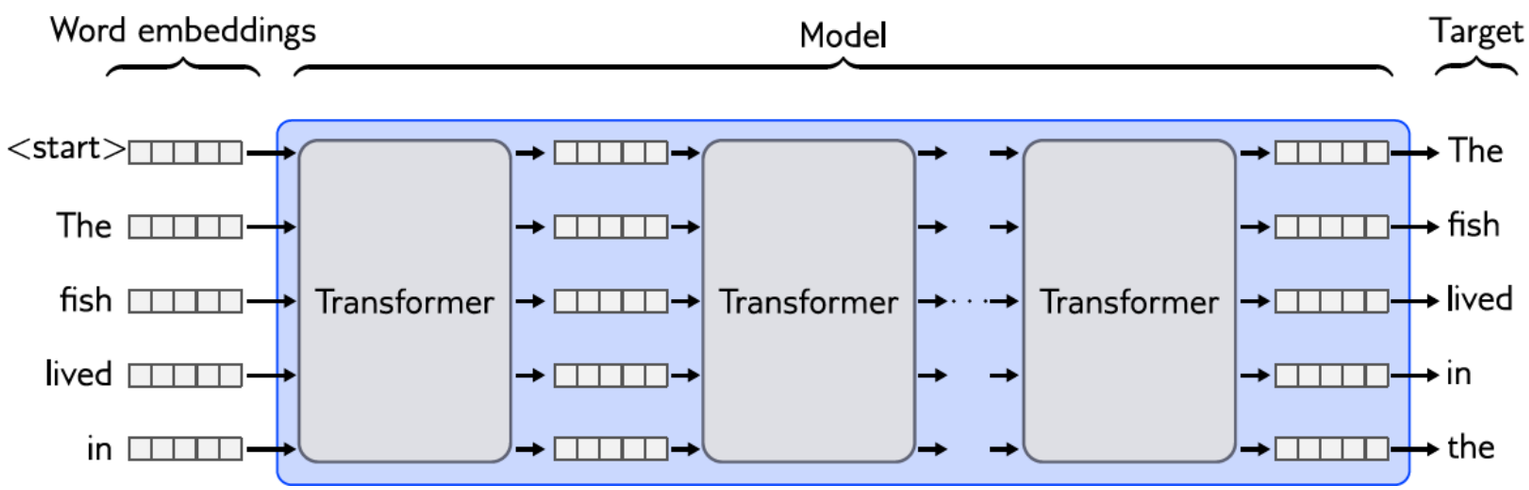
Decoder-only models: next-token prediction

- A decoder (GPT) is an **autoregressive** model of the token sequence:

$$p(t_1, \ldots, t_N) = \prod_{n=1}^{N} p\big(t_n \mid t_1, \ldots, t_{n-1}\big)$$

- Training = maximum likelihood = cross-entropy on the **next** token, at every position:

$$\mathcal{L} = -\sum_{n=1}^{N} \log p\big(t_n \mid t_{<n}\big)$$



- One forward pass trains **all N predictions in parallel** (teacher forcing). Generation: sample, append, repeat

Figure: S. J. D. Prince. Understanding Deep Learning. MIT Press, 2023, ch. 12 (udlbook.github.io)

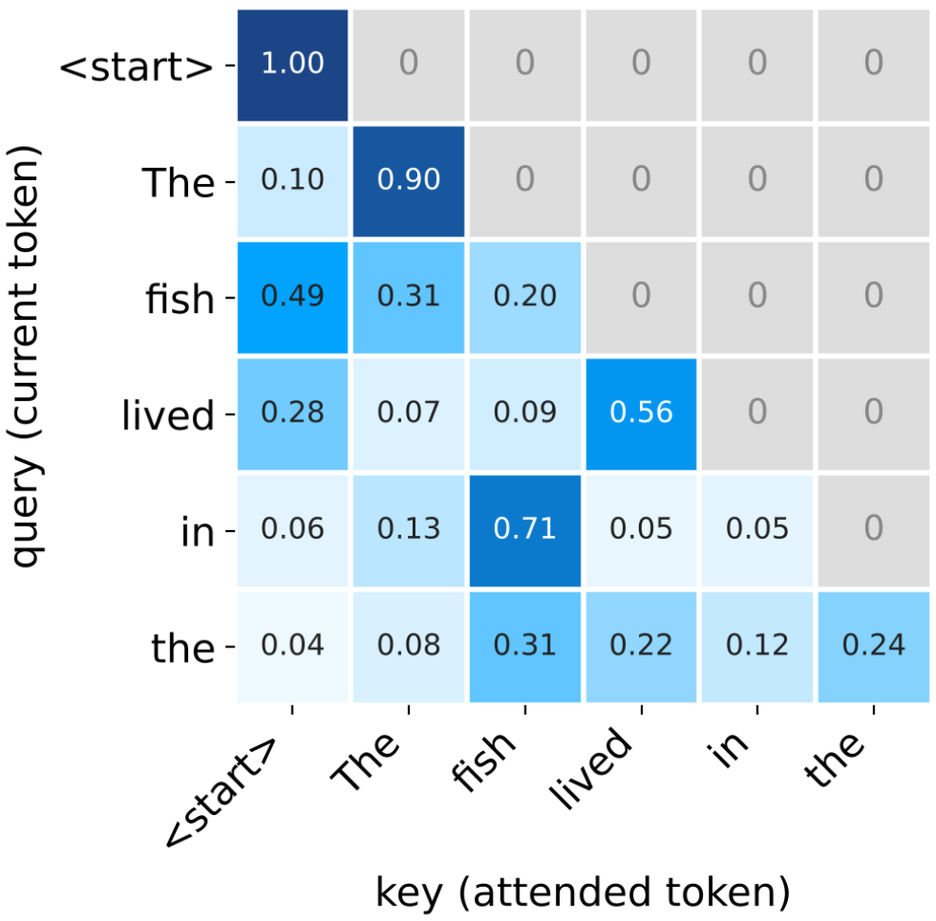
Causal (masked) self-attention

- Without a mask, position n would attend to n + 1, n + 2, … (the targets)
- Add $-\infty$ to the scores of **future** positions before the softmax:

$$M_{nm} = \begin{cases} 0 & m \le n \\ -\infty & m > n \end{cases}$$

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

- After the softmax those weights are exactly 0: token n only uses tokens 1 … n
- Same computation at training and generation time → no cheating, no mismatch
- Encoders (BERT) use no mask; decoders (GPT) always do

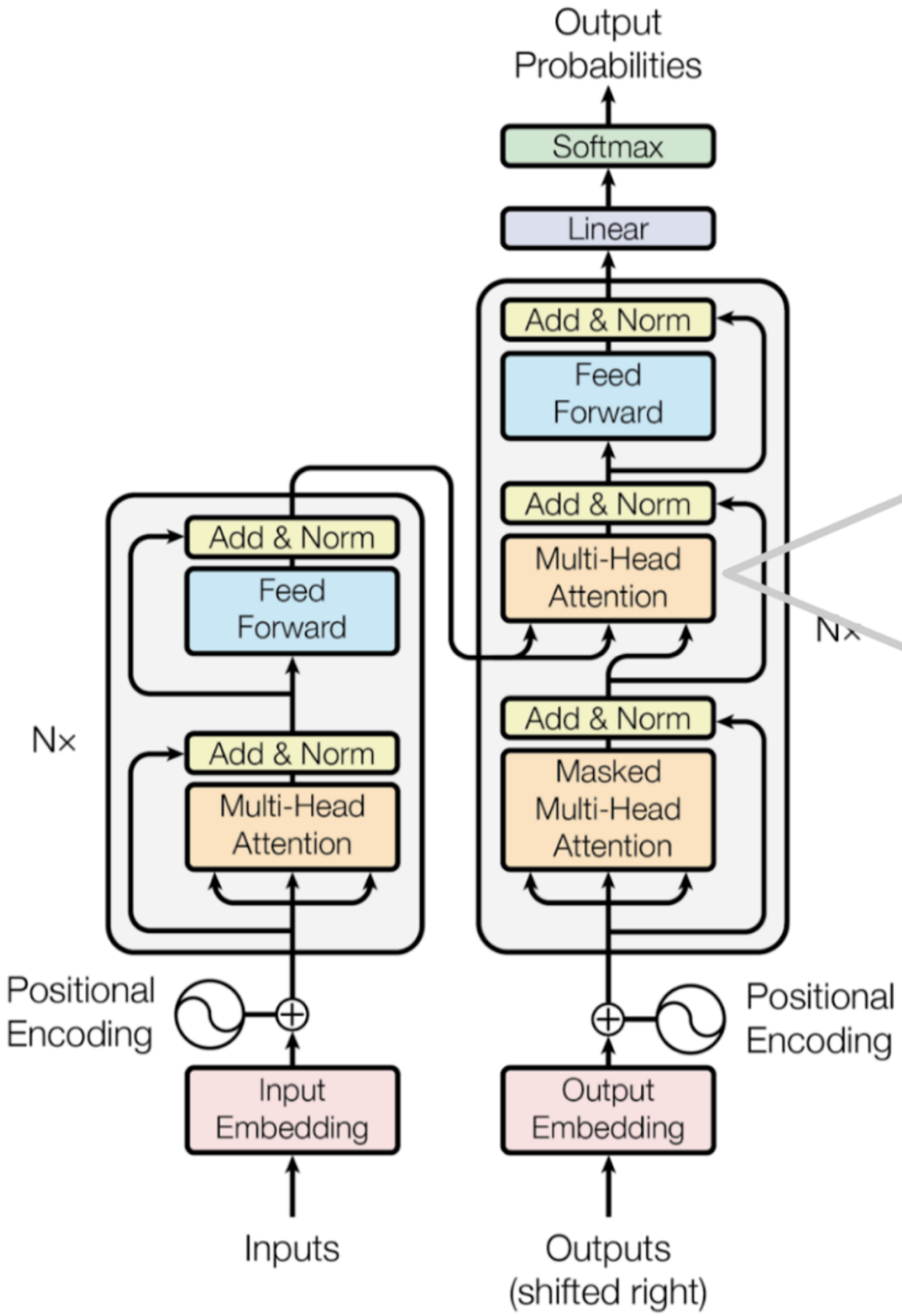


attention weights with the causal mask (illustrative)

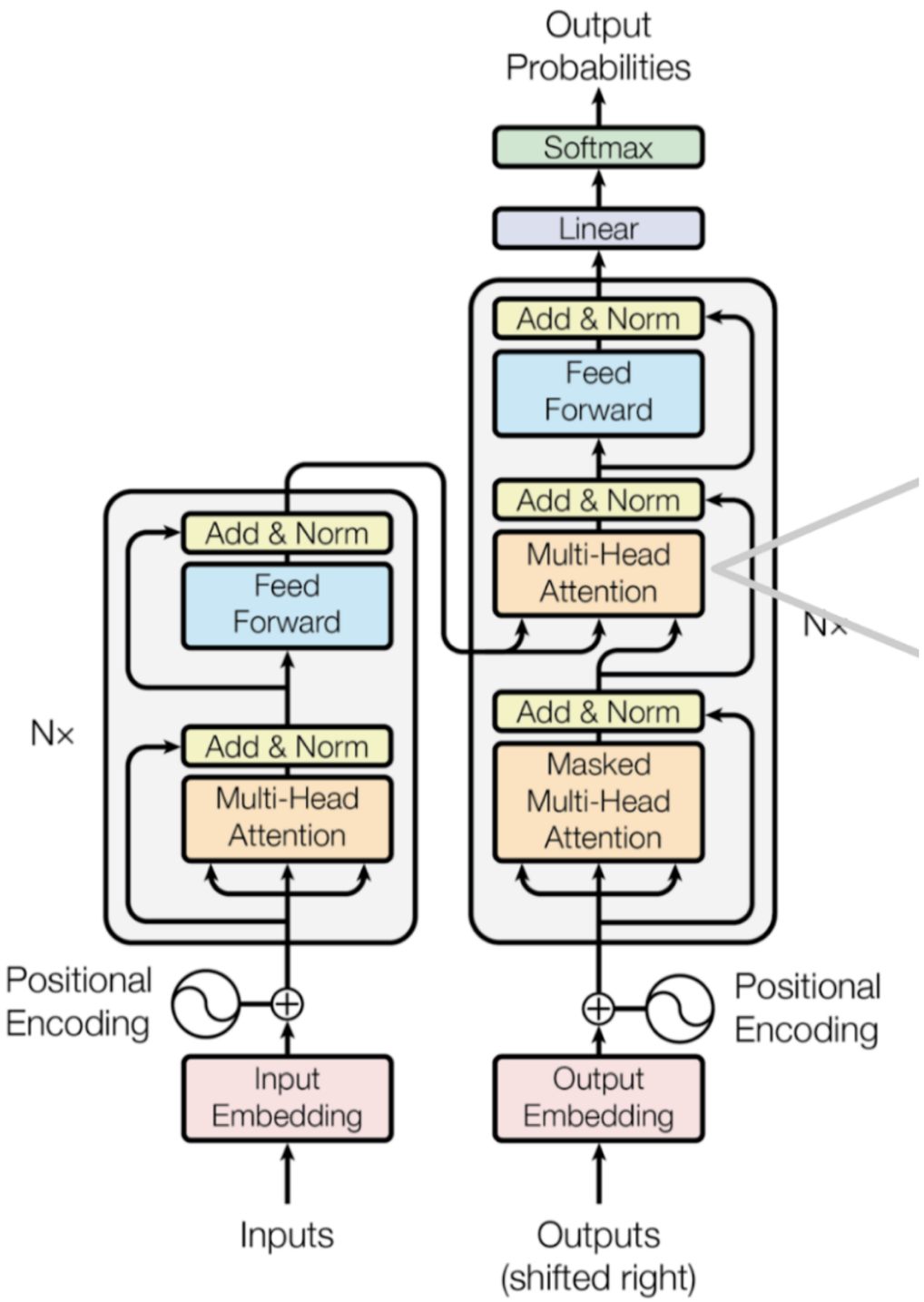
Self- vs cross-attention

- "***self-attention***":
  keys and values are produced from the same source as queries.

- "***cross-attention***":
  keys and values come from some other, external source (e.g. an encoder module), than the queries
- ***"Encoder-decoder"*** uses both:
  Queries come from the previous decoder layer, but Keys and Values come from encoder output

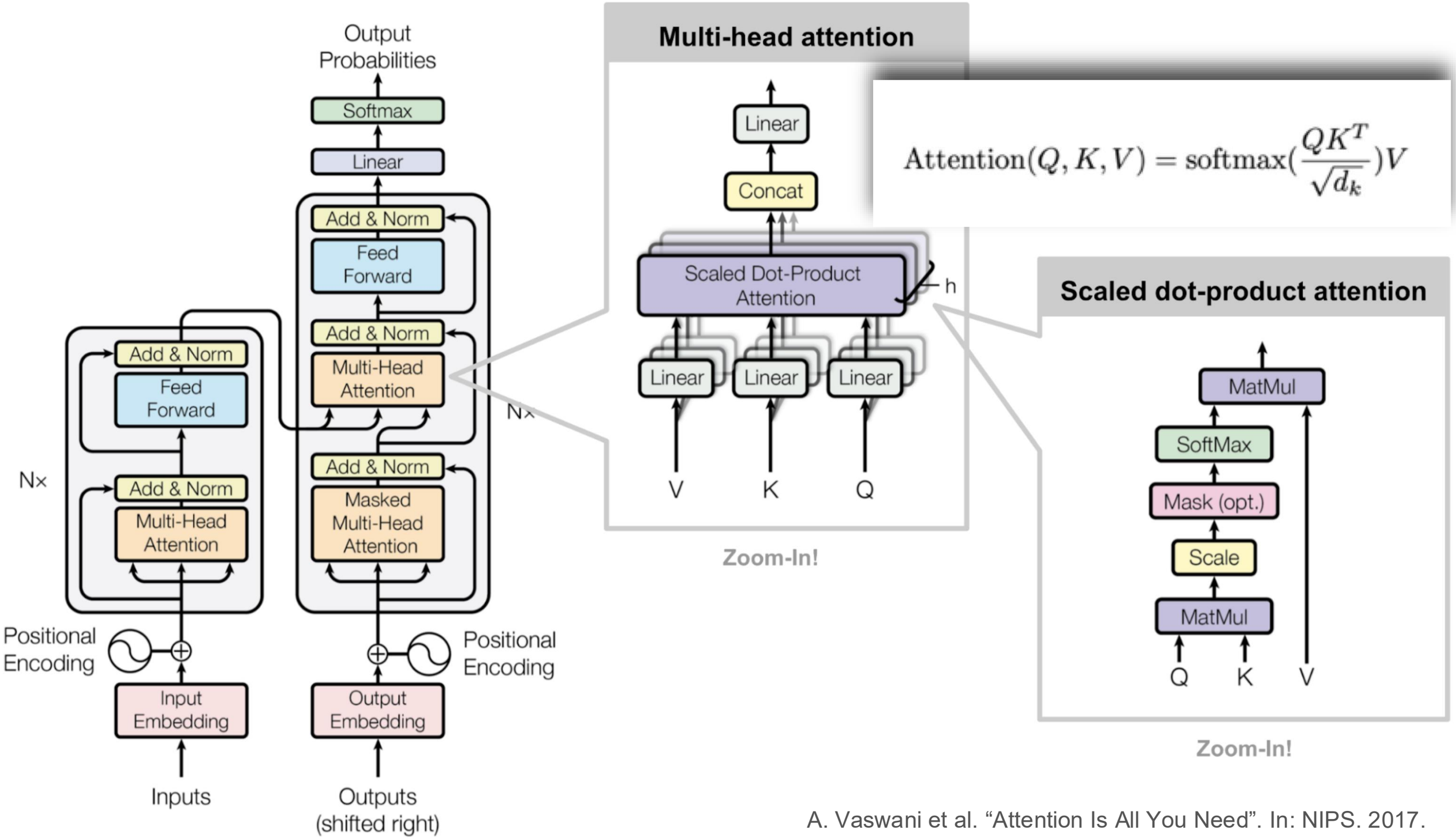


The original Transformer (encoder–decoder)



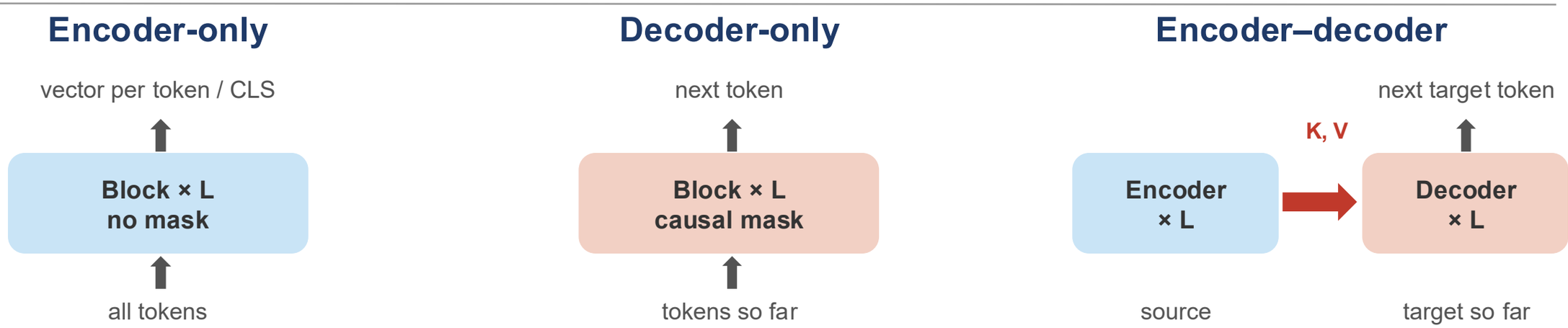
A. Vaswani et al. “Attention Is All You Need”. In: NIPS. 2017.

Zoom: multi-head attention



A. Vaswani et al. “Attention Is All You Need”. In: NIPS. 2017.

Three families, one building block



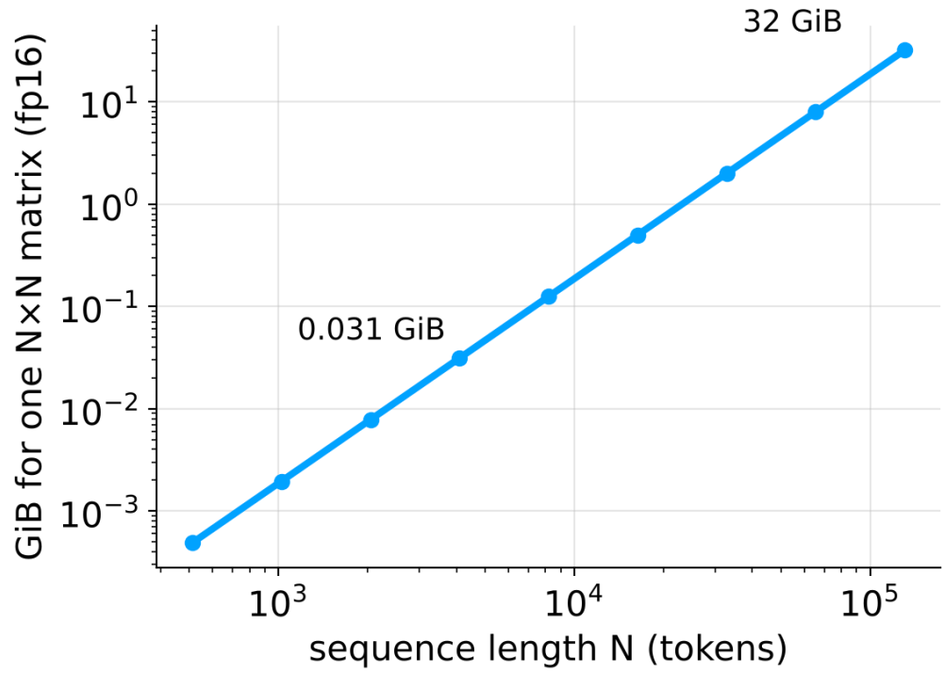
	Attention	Trained to	Typical use	Examples
Encoder-only	self, no mask	fill in hidden tokens	classification, retrieval, embeddings	BERT, ViT
Decoder-only	causal self	predict the next token	generation, chat	GPT, Llama, Claude
Encoder–decoder	self + causal self + cross	next target token, given the source	translation, captioning, speech → text	original Transformer, T5, Whisper

Slides 5–8 were already an encoder–decoder: **CNN encoder + cross-attention + LSTM decoder**. The Transformer replaces both sides with attention blocks.

Vaswani et al. 2017 · Devlin et al. (BERT) 2019 · Radford et al. (GPT) 2018 · Raffel et al. (T5) 2020 · Radford et al. (Whisper) 2023

The cost of attention

- QK^T has N × N entries: time $O(N^2 \cdot d)$, memory $O(N^2)$ per head and layer
  - RNN: O(N), but sequential. Attention: $O(N^2)$, but fully parallel
- **KV cache**: when generating token by token, the keys and values of past tokens do not change → store them, compute q, k, v only for the new token
  - memory grows linearly with the context length
- **Efficient attention**
  - FlashAttention: exact, tiled so the N × N matrix never sits in GPU memory
  - Sparse / sliding-window attention; linear and low-rank approximations
  - Multi-/grouped-query attention: heads share K, V → smaller KV cache



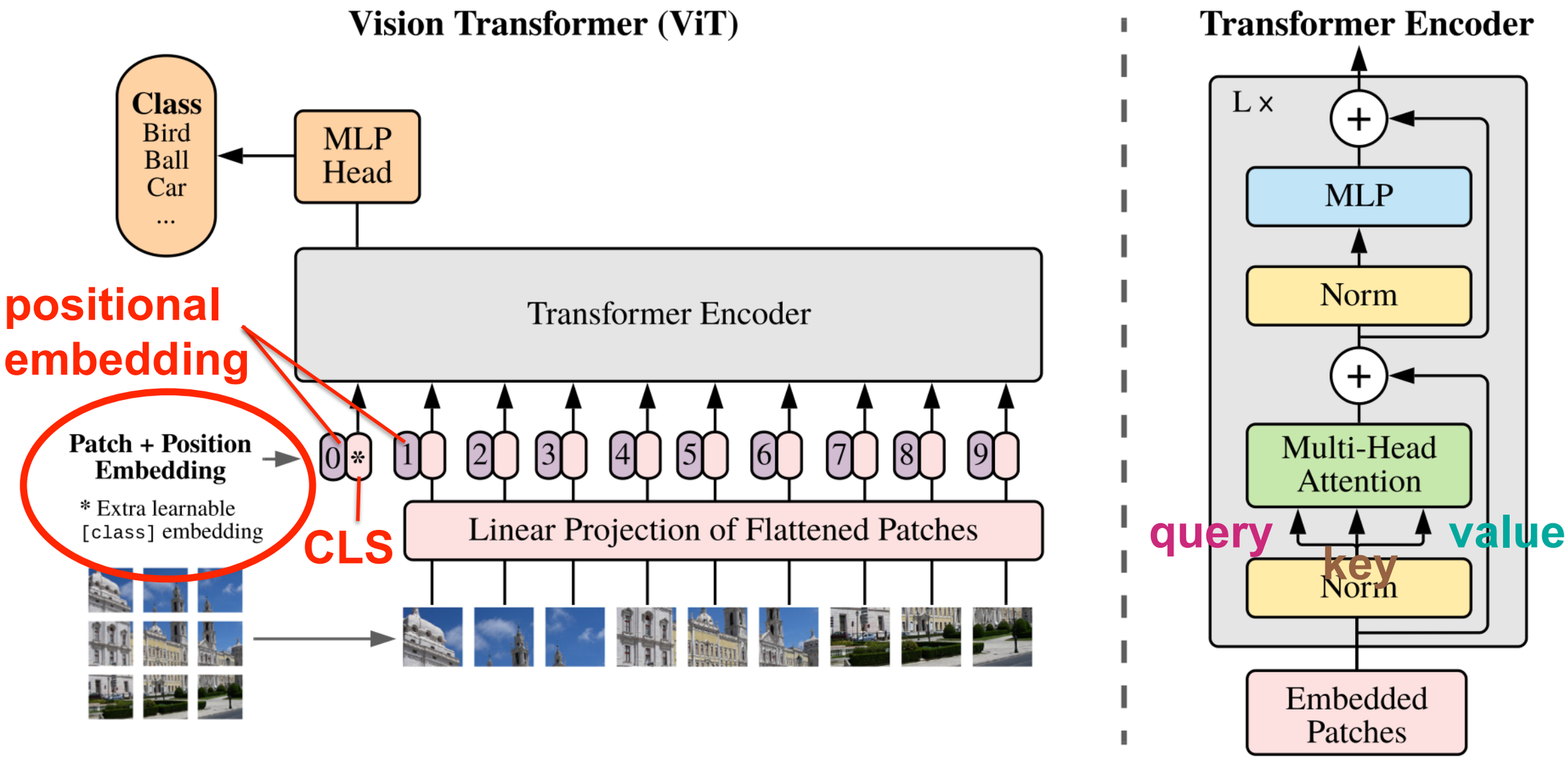
memory for one attention matrix (one head, one layer) if materialised

Go deeper → 62668 Acceleration for AI · 62653 Supercomputing · 62664 NLP with Language Models

T. Dao et al. FlashAttention. NeurIPS 2022 · J. Ainslie et al. GQA. EMNLP 2023 · I. Beltagy et al. Longformer. 2020

Visual Transformers

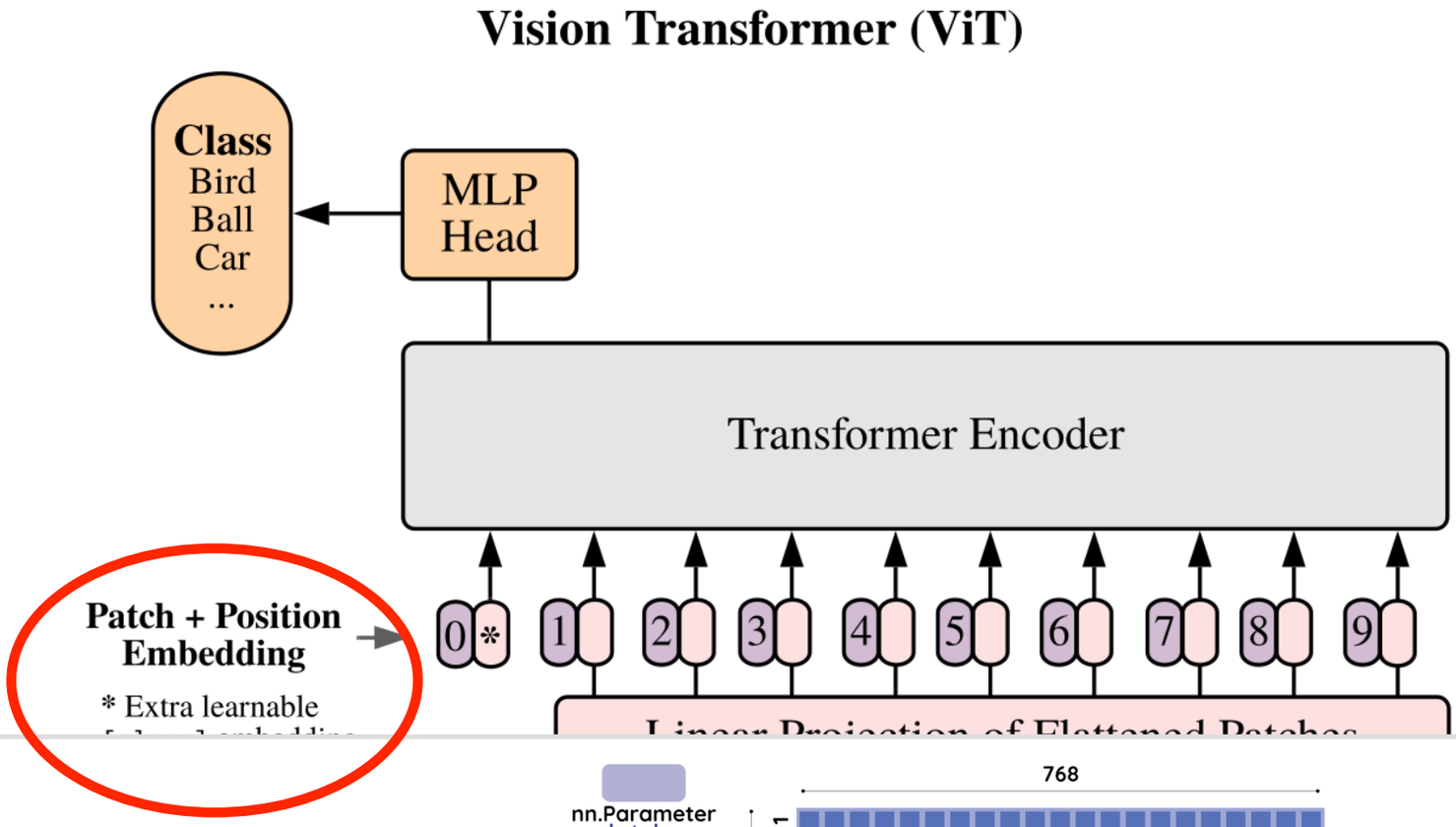
- The **Vision Transformer**: patches + positional encoding

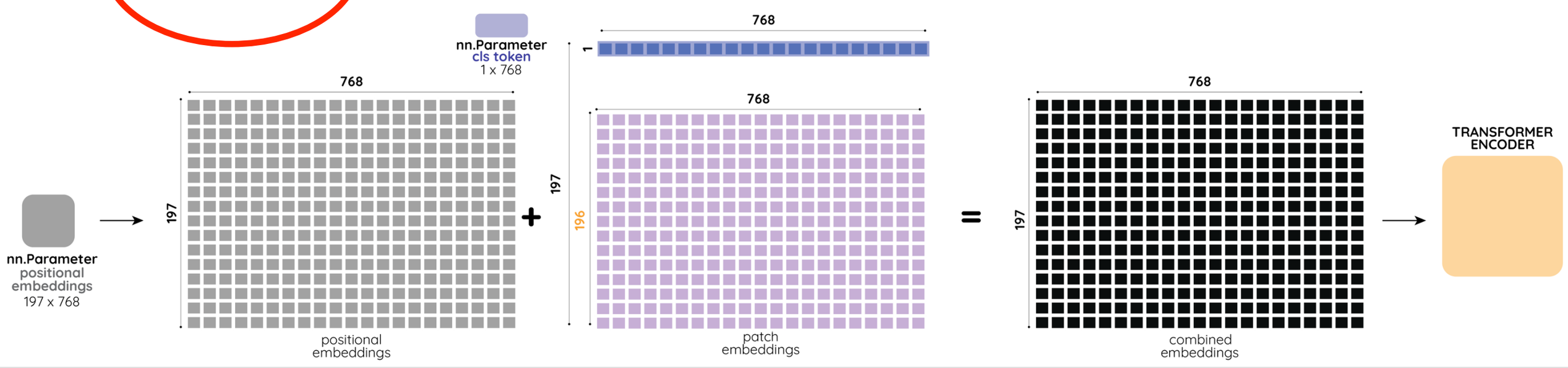


Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: ICLR. 2021.

Visual Transformers

- The **Vision Transformer**: patches + positional encoding



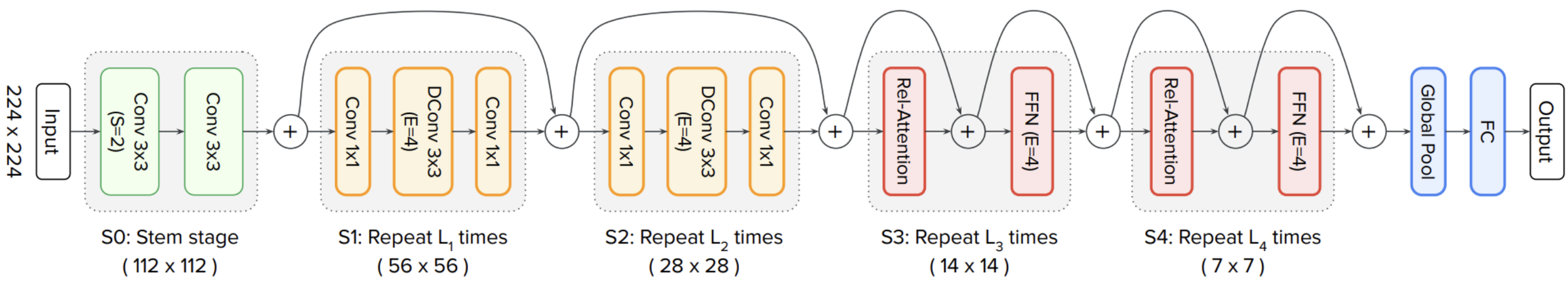


ViT: Differences with text

	Text (BERT)	Image (ViT)
Token	sub-word from the tokeniser	16 × 16 patch
Input to the embedding	integer index	768 pixel values
Embedding	lookup in Ω (D × |V|)	linear projection E (768 → D)
Learned?	yes, end-to-end	yes, end-to-end
Positions	learned, 1D	learned, one per patch (the 2D grid read in order)
Rest of the model	encoder + [CLS]	identical

Visual Transformers

- The **Vision Transformer:** later architectures used a conv layer instead of patches + linear projections for token embeddings
  - Transformers applied later, where the input is smaller
  - Input dimension to each block is downsampled

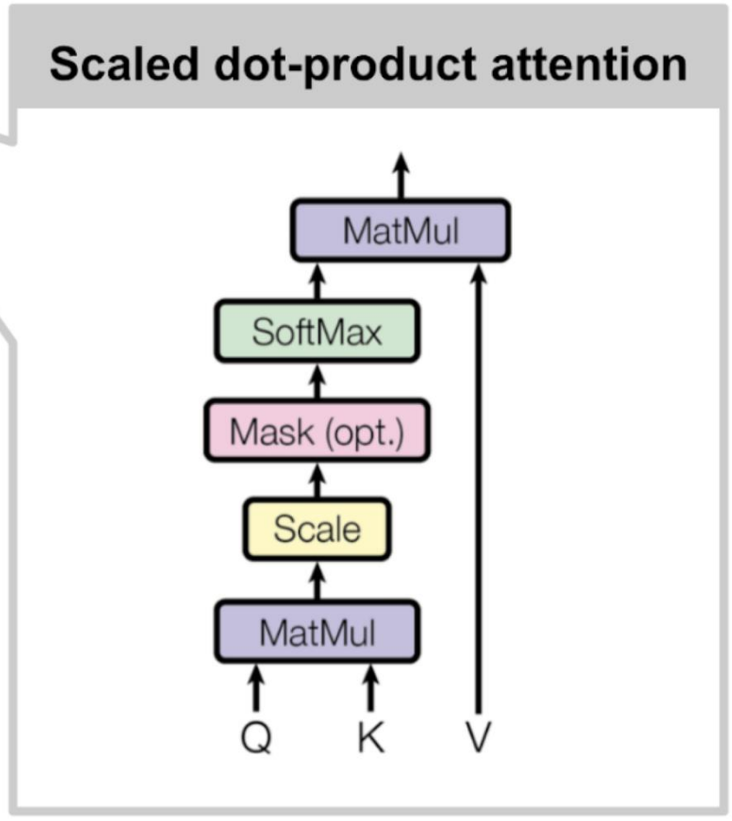


"CoAtNet: Marrying Convolution and Attention for All Data Sizes"

Exercise: A or B?

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

- **Transformers** - Attention



Zoom-In!

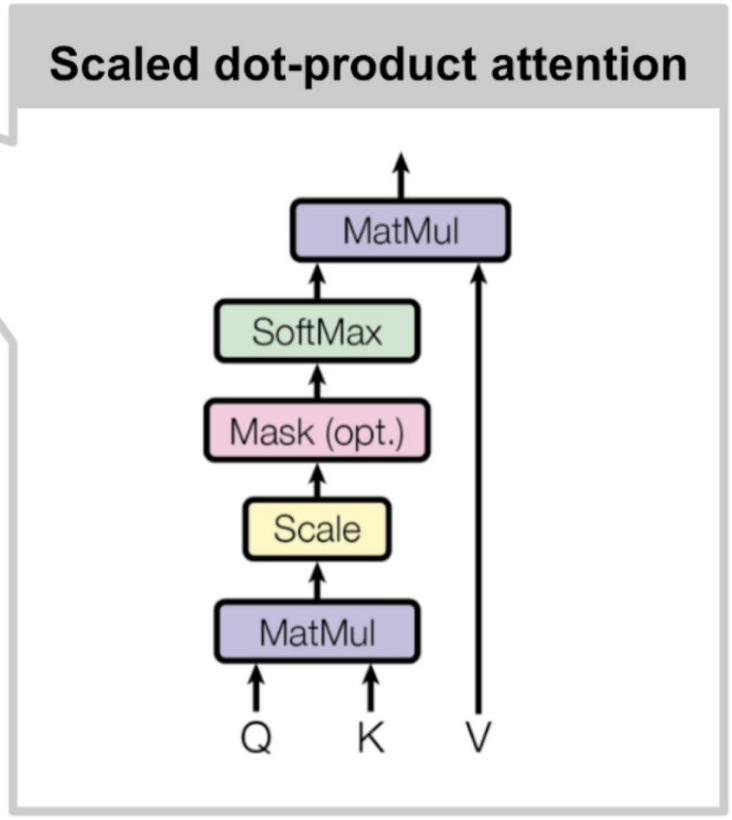
Exercise: A or B?

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Example: We are training an attention block to find a good representation for the given vocabulary (which contains "ambiguous" words).

Which matrix (A or B) provides a better encoding for the attention block we are training? To simplify:

- The 3 matrices are the same! **W_q = W_k = W_v**
- Compute only the new encoding obtained by the word "***apple***" given the context "*Eat red apple*"
- **Simplify the normalization step** by computing just **/3** as normalization step



Zoom-In!

```
Embedding INIT. (E):
Dimensions:
[Base_Tech-ness, Base_Nature-ness, Base_Food-ness]
apple : [1 1 1]
phone : [1 0 0]
eat   : [0 0 1]
tree  : [0 1 0]
red   : [0 0 0]
```

normalized_attention = K Q ***/ 3***
*new_V = normalized_attention * V*

1	0	0
0	1	0
0	0	1

Matrix **W_A**

1	1	0
0	1	1
1	0	0

Matrix **W_B**

Resources

- A. Vaswani et al. Attention is all you need. NeurIPS 2017
- S. J. D. Prince. *Understanding Deep Learning*, ch. 12 — udlbook.github.io
- J. Alammar. The Illustrated Transformer — jalammar.github.io
- Zhang, Lipton, Li, Smola. *Dive into Deep Learning*, ch. 11 — d2l.ai
- A. Dosovitskiy et al. An image is worth 16x16 words (ViT). ICLR 2021
- Blog on ViT, step by step — amaarora.github.io/2021/01/18/ViT.html

Slides adapted from A. C. Murillo (Univ. Zaragoza), MIT 6.869 (B. Freeman, P. Isola) and S. J. D. Prince (UDL)